FuzzPilot: Plateau-Triggered Recipe Validation for Structured Text Fuzzing

Zhiyi Yao
Qingdao University of Technology
202202030221@stu.qut.edu.cn

May 24, 2026

Abstract

Greybox fuzzers leave little room for expensive decision making in the mutation loop. FuzzPilot is a small controller around AFL++ that keeps those decisions off the hot path. At a coverage plateau, it takes a corpus snapshot, prepares candidate mutation "recipes", tests them in short isolated AFL++ runs, and promotes a recipe only when the validation run produces a positive reward. Recipes are data, not generated code: a native custom mutator consumes a compact JSON strategy containing operator weights, byte ranges, corpus-selection rules, and dictionary tokens. Candidate recipes can come from local rules or from a language-model agent, while Ghidra-derived constants and decompiled context provide target-specific hints.

This first public report is deliberately narrow. We evaluate the implementation on cJSON with N=5 repetitions for the two main arms, `baseline-afl` and `full-agent`, each with a 14,400 s budget. cJSON turns out to be a saturated target: vanilla AFL++ reaches the exposed 269-edge ceiling at a median of 2,524 s (time to first reach 269 edges; the distinct plateau-duration metric used in the next paragraph is 2,532 s). As a result, these experiments do not establish that language-model proposals improve coverage, nor do they support a generalization claim beyond cJSON.

Within that scope, the system has three useful properties. First, moving the controller off the hot path preserves throughput: `full-agent` has a median `execs_per_sec` about 1.06× the baseline. Second, `full-agent` has a descriptively shorter median plateau, 1,384 s versus 2,532 s for baseline, but the difference is not statistically significant at N=5 (Mann–Whitney p=0.42, A_{12}=0.68). Third, the validation gate behaved conservatively: it evaluated 20 model-proposed recipes and promoted none, because all candidate rewards were zero on the saturated corpus. Preliminary ablations suggest that the observed plateau reduction is more likely due to the controller's snapshot and restart machinery than to the recipe mutator or the model layer; a dedicated `controller-only` arm is left for the larger evaluation. This preprint is therefore best read as an auditable implementation report and a baseline for the ongoing non-saturated-target study, not as a claim that every component already improves fuzzing performance.

1 Introduction

Greybox fuzzers such as AFL++ are built around a simple constraint: the mutation loop must stay fast. On small parsers, a useful campaign can execute hundreds of thousands of test cases per second, so even a modest amount of per-input reasoning can dominate the run time. This constraint is awkward for systems that want to use richer target signals, such as decompiled constants, format-specific tokens, or language-model suggestions. Such signals are often useful, but they do not belong in the inner loop.

Recent LLM-assisted fuzzers explore several ways to manage that cost. Some invoke the model as an input generator or mutator during the campaign [14, 20, 23]; others move model work into a helper process [12]. G^{2}FUZZ [28] goes further by asking the model for a reusable Python generator only when the fuzzer stops making progress. That line of work suggests a practical rule for fuzzer design: expensive reasoning should happen at coarse intervals, and the product of that reasoning should be something the fuzzer can reuse cheaply.

FuzzPilot follows that rule for structured-text parsers. Instead of asking for executable generators, it works with mutation recipes: small data objects that describe how a native AFL++ custom mutator should bias its operators, byte ranges, token choices, and seed selection. Recipes can be written by local rules or proposed by a language-model agent. Either way, FuzzPilot does not trust a candidate immediately. At a plateau, it snapshots the current queue, tests candidate recipes in short AFL++ micro-campaigns, and promotes only a candidate that earns a positive reward. The design is intentionally conservative: an unhelpful candidate should cost a bounded validation run, not poison the main campaign.

Contributions. This report contributes the implemented FuzzPilot design and an initial audit of its behavior. The claims are intentionally limited: cJSON exercises the machinery end to end, but because the target saturates early, it does not show that every component is beneficial.

Scope and limitations (stated up front). Before enumerating contributions, we make the bounds of this preprint explicit. The limits are part of the result, not footnotes:

- **Single saturated target**: We evaluate on *only* cJSON, at N=5 repetitions for main arms and N=3 for ablations. cJSON is a *saturated target* where baseline AFL++ alone exhausts the reachable 269-edge ceiling at a median of 2,524 s (time to first reach 269 edges; the distinct plateau-duration metric used elsewhere is 2,532 s), leaving no headroom for the LLM proposal layer or micro-campaign gate to demonstrate coverage gains. We make *no* empirical claim about libpng, libxml2, sqlite3, openssl, or any other target.

- **LLM layer untested**: The micro-campaign gate evaluated 20 LLM-proposed recipes on cJSON and promoted zero, because target saturation leaves reward R=0 on every candidate (§6.4). The gate's intended property—distinguishing good recipes from bad on non-saturated targets—remains *empirically undemonstrated*. A non-saturated-target evaluation is the most important single piece of future work.

- **Statistical power insufficient**: At N=5 (main) and N=3 (ablations), no pairwise plateau or coverage difference reaches conventional significance thresholds (p<0.05). We report descriptive point estimates and effect sizes (A_{12}), not significance claims. Formal rank-sum tests require $N \geq 20$ for main arms and $N \geq 10$ for ablations [10].

- **Attribution incomplete**: Ablation experiments (§6.5) suggest the plateau reduction is driven primarily by the controller's plateau-detection and corpus-reorganization machinery, not by the recipe-guided mutator or LLM layer. However, a `controller-only` ablation (plateau detector + corpus snapshot only, no mutator, no LLM, no Ghidra) is *not* in the preprint matrix; without it we cannot fully separate "controller machinery" from "default rule recipe". This ablation is deferred to the venue version.

- **Fairness baseline caveat**: An E4 fairness arm shows that AFL++ with `cmplog` (AFL++'s comparison-operand logging pass) instrumentation reaches 270 edges (vs FuzzPilot's 269)

without shortening the plateau (§6.5), indicating the two techniques operate on complementary axes (wall-time vs absolute coverage).

Taken together, these constraints mean that this version should be read as an implementation report and a cJSON proof-of-concept. The larger non-saturated-target matrix, higher repetition counts, and the missing `controller-only` ablation are the next steps. §8 gives the full list of threats to validity. With these bounds explicit, the contributions are:

C1 A **data-as-recipe** intermediate representation that replaces generated helper code with a schema-validated JSON description of mutation strategy — a weighted distribution over seven mutation operators, plus focus/protect byte ranges, dictionary tokens, and a corpus selector — dispatched by a native AFL++ custom mutator. This side-steps the runtime-safety concern of executing generated code, makes proposals cacheable and version-able, and lets the mutator hot path remain pure native code at AFL++ speed (§4).

C2 A **plateau-triggered controller** that detects coverage stalls via a moving-window edge-discovery rate, takes a corpus snapshot at plateau time, and can trigger intervention strategies (model proposals, corpus reorganization, or rule-based recipes). The controller operates off the hot path and does not block AFL++ workers. Ablation experiments (§6.5) suggest this machinery is the primary driver of the observed plateau reduction on cJSON, though a `controller-only` ablation to isolate its contribution is deferred to the venue version (§8).

C3 A **micro-campaign validation gate** that scores each candidate recipe in an N-second isolated AFL++ run over a corpus snapshot and promotes only those with positive reward (new edges, paths, or crashes). The gate gives the controller empirical evidence about each proposal before it enters the main loop. On cJSON, the gate evaluated 20 LLM-proposed recipes and promoted zero due to target saturation (§6.4); its intended property—distinguishing good recipes from bad—remains untested and requires a non-saturated target.

C4 A **static-analysis context channel** built on Ghidra headless that extracts magic tokens, comparison constants, branch constraints, and decompiled C from stripped binaries. The signal feeds both the agent blackboard (for LLM prompts, when enabled) and the AFL dictionary (for direct mutation guidance). The ablation E2c (§6.5) jointly quantifies the loss of both consumers; isolating Ghidra's advantage over source-level alternatives is deferred to the venue paper (§8).

The artifact also includes an auditable decision trail covering blackboard state, proposal generation, recipe emission, micro-campaign reward, and promotion outcome, plus native Linux/x86_64 run metadata from the experiment host.

Roadmap. §2 surveys the design space and positions FuzzPilot relative to G^{2}FUZZ and the in-loop line of work. §3 presents the architecture. §4 details the recipe representation and contrasts data-as-recipe with code-as-generator. §5 specifies the plateau detector, the Ghidra context channel, and the micro-campaign protocol. §6 reports preliminary results on cJSON. The per-run decision trail (events log, recipe proposals, micro-campaign reward, promotion outcome) is recoverable from `results/.../runs/p1_e1_cjson_full-agent_r*/events.jsonl`. §7 relates FuzzPilot to the broader LLM-fuzzing literature. §8 states limitations and threats to validity.

2 Background and the Off-Hot-Path Paradigm

2.1 The AFL++ mutation loop and its throughput budget

AFL++ implements a deterministic-then-havoc mutation loop [8]: each cycle takes an input from the queue, applies a sequence of bit/byte mutations, executes the target via `fork+execve` or a persistent harness, and updates the coverage bitmap. On small parsers such as cJSON our 4-core x86_64 host sustains roughly 13,000 executions per second per worker. Any redesign that interposes a non-trivial computation on this loop caps end-to-end throughput at the rate of that computation: inserting a single 100 ms LLM call per mutation drops throughput by three orders of magnitude. Existing LLM-in-the-loop designs accept this cost in exchange for semantic input quality.

2.2 In-loop LLM fuzzers

LLAMAFUZZ [20] fine-tunes an LLM on seed-mutation pairs and queries it as part of the mutation step; the model output is then handed to AFL++. FuzzCoder [14] casts mutation as a sequence-to-sequence task and predicts mutation positions and strategies per input. Fuzz4All [23] drives an LLM as the primary input generator and mutation engine across multiple input languages. The hybrid design of Lin et al. [12] reduces the per-mutation cost by isolating LLM work in a helper fuzzer at roughly 250 queries per hour, with a syntax-validator-and-repair filter, while a master fuzzer runs AFL++ at full speed. None of these designs remove the LLM from steady-state fuzzing.

2.3 The off-hot-path paradigm

G^{2}FUZZ [28] invokes the model *only* when the underlying fuzzer fails to find new coverage, and it does so by asking the model to synthesize a Python input generator. The generator is executed to produce a batch of candidate inputs, which AFL++ then mutates as usual. The model contributes strategy at coarse temporal granularity; mutation throughput remains native. G^{2}FUZZ shows this paradigm is practical on 34 non-textual input formats and outperforms AFL++, Fuzztruction, and FormatFuzzer in coverage and bug-finding. We treat the off-hot-path placement as a starting *contract* that FuzzPilot inherits rather than a contribution we claim.

2.4 Why structured text needs a different intermediate

G^{2}FUZZ's intermediate representation is *Python code*: the model emits a generator script and the fuzzer executes it. Two properties of the non-textual binary setting make this choice attractive: (i) binary formats have rich constructor APIs (e.g., `PIL.Image.new` for JPEG/TIFF) that make generator scripts compact, and (ii) the generated bytes are opaque, so AFL++'s bit/byte mutators are appropriate downstream.

Structured text parsers exhibit the opposite properties. The relevant edits at the syntactic level — inserting a key, replacing a literal with an interesting value, splicing two well-formed sub-trees, applying a dictionary token — are easier to express as small *data* (an opcode, a target offset, a replacement literal) than as code. Furthermore, running LLM-emitted code at runtime carries non-trivial safety risk (network calls, file I/O, unbounded loops) that a schema-validated data structure avoids. §4 develops this contrast in detail.

2.5 Open design questions for structured text

Within the off-hot-path paradigm, three design questions remain open for structured text parsers and motivate the rest of the paper:

- **Q1: Intermediate representation.** Code-as-generator or data-as-recipe? (§4)
- **Q2: Proposal validation.** How does the controller decide whether a candidate proposal is worth a multi-hour main-loop commitment? (§5)
- **Q3: Static context source.** When the target's source is unavailable, where does the structured context come from? (§5.2)

3 FuzzPilot Design

3.1 Architectural contract

The architecture (Figure 1) is organized around a single invariant: *no external reasoning call appears on any code path executed by AFL++ during per-mutation work.* Model calls occur only when (a) the plateau detector marks a window as stalled, or (b) a micro-campaign has completed and a promotion decision is needed. The mutator never holds an open connection to the optional model client; it consumes recipes from a local store that contains only validated data.

This contract is shared with G^{2}FUZZ; we adopt it as the baseline. The remainder of this section describes how FuzzPilot specializes the contract for the structured-text-parser domain.

3.2 Component overview

Plateau detector. A coverage-window monitor that flags a plateau when fewer than θ_e new executions and fewer than θ_p new paths accumulate over W seconds. Hyperparameters are pinned per target (§5). On a plateau the detector freezes a snapshot of the queue and writes it to the blackboard.

Blackboard. A JSON document holding the plateau snapshot, recent fuzzer_stats, and the Ghidra-derived static context for the target binary (§5.2). The blackboard is the LLM's only view of the campaign state.

Proposal workers. A small set of role-specialized prompt templates (coordinator, plateau-diagnosis, scheduler, cmp-hint, dictionary, format, mutator, corpus, crash-triage). Each agent reads the blackboard and proposes a recipe or a sub-recipe; the coordinator assembles a final proposal. We do not claim the role split itself as a novelty in this preprint; it is an engineering convenience and is ablated jointly with the other control-path components.

Recipe store. A versioned, schema-validated collection of recipes proposed, evaluated, and (in some cases) promoted. The custom mutator reads from this store at recipe-load time only.

Micro-campaign runner. Launches an isolated N-second AFL++ run with a candidate recipe installed, computes a reward over its delta in edges and paths, and reports back. The micro-campaign shares the target binary with the main run but uses a private working directory and a snapshot of the corpus.

Custom mutator. A native AFL++ mutator that dispatches on the seven recipe operations (`BitFlip`, `OverwriteRange`, `InsertToken`, `Arith`, `Splice`, `DeleteBlock`, `DictionaryOverwrite`; the full enumeration is in §4.2). The mutator is the only on-the-hot-path component that touches recipes; its cost profile is reported in §6 (RQ2, F5).

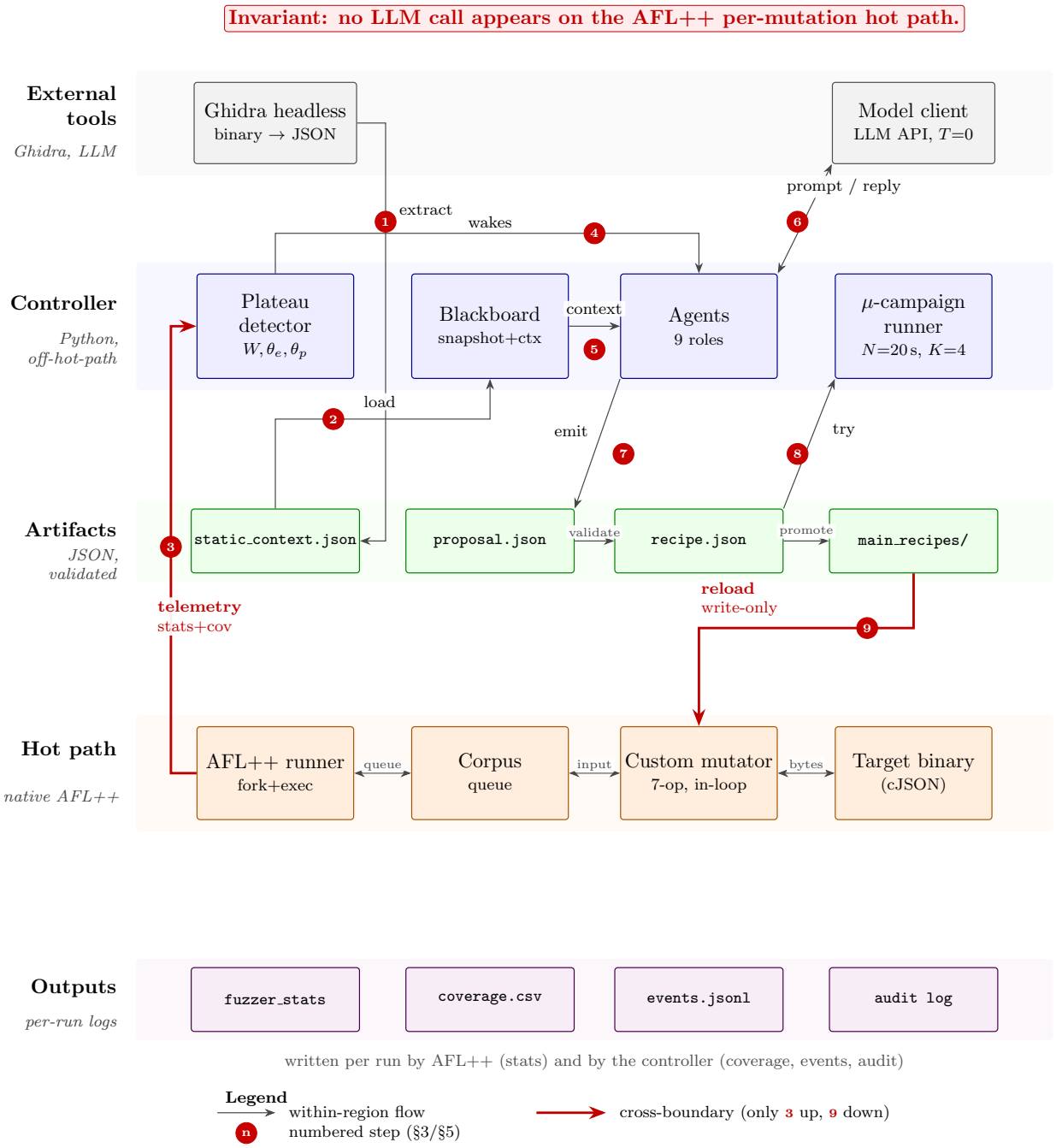

Figure 1: FuzzPilot end-to-end architecture. Five horizontal stripes separate **external tools** (Ghidra, optional model client), the off-hot-path **controller** (plateau detector, blackboard, agents, micro-campaign runner), persisted **artifacts** (`static_context.json`, `proposal.json`, `recipe.json`, `main_recipes/`), the native **hot path** (AFL++ runner, corpus, custom mutator, target binary), and per-run **outputs** (`fuzzer_stats`, `coverage.csv`, `events.jsonl`, audit log). Only two edges cross stripes (drawn in red): telemetry up (Step 3) and recipe promotion down (Step 9). No per-mutation hot-path code ever touches the optional model client. Numbered edges (1)–(9) are walked through in §3 and §5.

4 Recipe Abstraction: Data, Not Code

4.1 Recipe schema

A recipe carries two layers of state that the implementation keeps in lock-step. The high-level form, `SeedMutationStrategy` (`include/fuzzpilot/mutation/strategy.hpp:38--49`), is what the agent emits and the controller serializes; it is lowered to a compact binary form, `CompactRecipe` (`mutators/fuzzpilot/recipe_cache.hpp:45--58`), before the custom mutator loads it. The two share the following core fields:

- `id`: stable identifier; lets repeat recipes deduplicate across plateaus and across runs.
- `selector`: which subset of corpus elements the recipe applies to. A selector is a (*mode*, *key*) pair where *mode* is one of `mode` / `seed_id` / `seed_hash` / `family` and *key* is the matching string.
- `priority` and `ttl_sec`: scheduling weight and time-to-live for the recipe store.
- `operator_weights` (in compact form: `weights`): a probability distribution over the fixed 7-operation vocabulary (§4.2). The mutator samples *one* op per call by these weights; the recipe is not an ordered program.
- `focus_ranges` / `protect_ranges`: half-open byte ranges $[a, b)$ where the mutator is preferred (respectively forbidden) to write. Derived from agent inference and from the Ghidra-extracted `cmp_constants` and `branch_constraints` (§5.2).
- `dictionary_tokens` (compact: `tokens`): literal byte sequences that the `InsertToken` and `DictionaryOverwrite` ops draw from. Populated from Ghidra `magic_tokens` and from agent-proposed tokens.
- `expected_signal`: the agent's prediction of what kind of coverage gain this recipe should produce. Used by the audit log only; never consulted by the mutator.
- `fields` (compact only): per-field overrides for structured parsers, e.g. "JSON object values" or "PNG IHDR block".

Recipes that fail JSON-schema validation are discarded before they reach the recipe store. The mutator never parses free-form model output — it only loads `CompactRecipe` entries from a local binary cache.

4.2 Operation vocabulary

The op vocabulary is small (seven ops) and closed (`mutators/fuzzpilot/recipe_cache.hpp:9--17`):

- `BitFlip` — single-bit flip at a chosen offset.
- `OverwriteRange` — write random bytes into a contiguous range.
- `InsertToken` — splice a token from `dictionary_tokens` at a chosen offset.
- `Arith` — arithmetic on a numeric field (add / subtract / multiply / divide).

Listing 1: Example recipe (cJSON, focus on object/array boundary tokens).

```
{
  "id": "p1_e1b_r03_plateau_07",
  "selector": {"mode": "seed_hash", "key": "9c4f...a7b1"},
  "priority": 3,
  "ttl_sec": 1800,
  "operator_weights": {
    "InsertToken":          0.35,
    "DictionaryOverwrite": 0.25,
    "Splice":               0.15,
    "OverwriteRange":       0.10,
    "BitFlip":              0.10,
    "Arith":                0.03,
    "DeleteBlock":          0.02
  },
  "focus_ranges":    [[0, 1], [42, 64]],
  "protect_ranges": [[16, 20]],
  "dictionary_tokens": ["{", "}", "[", "]", "\"", "true", "null"],
  "expected_signal": "exercise object/array nesting boundary"
}
```

- `Splice` — replace a sub-range with bytes lifted from another queue element.
- `DeleteBlock` — excise a contiguous range.
- `DictionaryOverwrite` — overwrite bytes at a chosen offset with a dictionary token.

Each call to the mutator's fuzz function invokes `recipe.choose_operator(rng)` to sample one op by `operator_weights` and dispatches it through a 7-arm switch (`mutators/fuzzpilot/mutator_core.cpp:292--318`). There is no opcode interpreter and no nested control flow: the mutator's work per mutation is bounded by the cost of the chosen op, which is the same order of magnitude as AFL++'s native havoc operators. This keeps the throughput cost of recipe dispatch within the budget we report in §6.3.

4.3 Example recipe (illustrative)

Listing 1 shows a recipe an agent might emit after inspecting a JSON parser's Ghidra context (an opening brace `0x7b` expected at offset 0; `strcpy` call site detected at offset `0x108`). The mutator loads the recipe from cache and, on each call, samples an op by `operator_weights` and applies it inside `focus_ranges`, drawing tokens from `dictionary_tokens` when an `InsertToken` or `DictionaryOverwrite` op is chosen. The agent never re-enters the loop during mutation.

4.4 Recipe lifecycle

The lifecycle (Figure 2) is: **(1) Proposal** from one or more agents; **(2) Schema validation** (discard malformed proposals); **(3) Micro-campaign** isolated N-second AFL++ run with the proposal installed; **(4) Reward** computed over Δedges and Δpaths; **(5) Promotion** of top-K proposals to the main recipe store, others archived to the audit log.

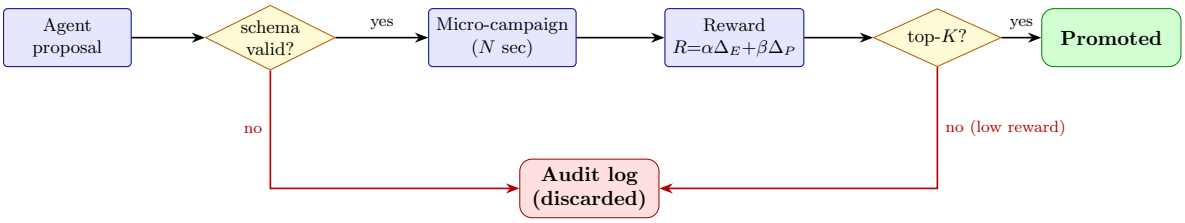

Figure 2: Recipe lifecycle. Each LLM proposal is first schema-validated, then evaluated in an N-second isolated micro-campaign, and only the top-K by reward are promoted to the main run. Discarded recipes are kept in the audit log.

4.5 Why data, not code? Contrast with code-as-generator

G^{2}FUZZ's input-generator approach is the natural intermediate for non-textual formats, but in the structured-text setting data-as-recipe has three concrete advantages.

Runtime safety. Executing LLM-emitted Python at fuzz time exposes the harness to a class of risks that schema-validated data does not: import side effects, file I/O, network calls, and unbounded loops are all reachable from a free-form generator. The recipe dispatcher has no I/O capability beyond reading bytes from the mutated buffer and writing bytes back.

Cacheability and version control. A recipe is a small ($\leq$ 2 KB) JSON document with a stable schema; thousands of recipes can be diffed, deduplicated, indexed by hash, and re-evaluated without re-prompting the model. A generator script is larger, depends on Python and library versions, and is awkward to de-duplicate or diff.

Schema-validatable in finite time. The recipe schema is a closed grammar; validation is a JSON-schema check. A generator script is Turing-complete and cannot be statically validated for behavior; G^{2}FUZZ relies on a syntactic-validator plus a sandbox to compensate.

We do not claim data-as-recipe is universally superior; in the non-textual setting it is plausibly less expressive. The claim is narrower: for structured text parsers, data-as-recipe yields a safer and more reusable intermediate than code-as-generator, and our evaluation (§6) reports its concrete cost (`fp-active` vs `fp-empty`).

5 Plateau Detection, Ghidra Context, and Micro-Campaign

5.1 Plateau detection

The plateau detector inspects a sliding window of W seconds and flags a plateau when, within that window, both (a) fewer than θ_e new executions and (b) fewer than θ_p new paths have accumulated. The three values are pinned per target in the run configuration (`experiments/targets/cjson/config.yaml`); for cJSON we use $W = 10$ s, $\theta_e = 50$ execs, $\theta_p = 1$ path. In each of the 5 `full-agent` runs the detector fired exactly one plateau cycle within the 14,400 s budget (corroborated by the `corpus_snapshot` event count of 1 per run in `events.jsonl`); cJSON is small enough that one plateau is sufficient to use the productive-then-plateau window once. On a flagged plateau the detector freezes a queue snapshot, writes it to the blackboard, and signals the coordinator agent.

5.2 Ghidra-derived static context

Extraction pipeline. Once per target binary, FuzzPilot launches Ghidra `analyzeHeadless` in an ephemeral project directory (`<run_dir>/ghidra_project`, automatically deleted on exit) and runs

a custom post-script, `FuzzPilotGhidraExtract.java`. The script writes `static_context.json` with the following schema:

- `functions[]`: top-20 risky functions, each annotated with name, address, size, list of risky calls, and a danger score. The score weights direct calls to known unsafe primitives: $10 \times \#\{$`gets`, `system`, `popen`$\}$ $+5 \times \#\{$`strcpy`, `strcat`$\}$ $+4 \times \#\{$`sprintf`$\}$ $+3 \times \#\{$`malloc`, `free`, `memcpy`, `memmove`$\}$ $+2 \times \#\{$`printf`$\}$.
- `magic_tokens[]`: up to 150 candidate dictionary entries — defined strings recovered from Ghidra's `getDefinedData()` pass and 4-byte ASCII constants, in both endiannesses.
- `cmp_constants[]`: up to 100 scalar operands appearing in comparison instructions, useful for the `numeric_interesting` op of the recipe schema.
- `branch_constraints[]`: up to 150 tuples of the form (address, condition mnemonic, comparison mnemonic, operand 1, operand 2), recovered by a 4-instruction lookahead preceding each conditional jump.
- `decompiled_logic{}`: the decompiled C source for the top-5 risky functions, truncated to 4 KB each, for prompt-readable context.
- `structs`: target-defined struct layouts when available.

Two consumers, one extraction. The same `static_context.json` feeds two downstream components without re-entering the model:

1. **Agent blackboard.** The JSON is merged into the blackboard's `static_analysis_context` field and read by every agent prompt during plateau handling. The LLM sees the ranked risky functions and decompiled C as natural-language context, alongside the queue snapshot and recent telemetry.
2. **Micro-campaign AFL dictionary.** The `magic_tokens` and `cmp_constants` are also converted into an AFL-format dictionary file (`static_generated.dict`) that the micro-campaign passes to AFL++ via `-x`. This puts static-analysis-derived tokens directly in the hands of the in-loop mutator, with no per-mutation LLM call.

This dual-consumer design is a deliberate choice: the LLM uses the context for high-level proposal, while AFL++ uses it for low-level token-level mutation, and both share a single, cacheable extraction.

Caching. The first run of a campaign emits `<run_dir>/static_context.json` and, if no prior cache exists for the binary, renames it to `base_intelligence.json`. Subsequent runs on the same binary load the cached JSON rather than re-running Ghidra. A 4-KB decompilation cap and the top-20 / top-150 / top-100 limits keep the artifact under 1 MB even for large targets.

Failure handling. If `analyzeHeadless` exits non-zero, FuzzPilot writes an error envelope into `static_context.json` rather than an empty object. Downstream code sees the error explicitly, and the audit log records the failure so a re-run can replay the same condition. The plateau handler still proceeds (with an empty static context); the LLM's proposals are noted as "static context unavailable."

Why Ghidra (and an honest caveat). We choose Ghidra over LLVM IR or angr for three operational reasons: (a) Ghidra accepts stripped binaries without source code or rebuild flags, broadening applicability; (b) its decompiled C output is closer to what a human reverse engineer would feed an LLM, and our prompts are trained on that style; (c) Ghidra headless is a stable production tool with predictable resource use, suitable for unattended fuzz campaigns. The channel is an **enabling component** of the architecture, not a free-standing novelty: on our open-source targets, an LLVM-IR or angr-based pipeline could in principle supply the same fields. Ablation E2c (§6.5) quantifies the contribution of the entire channel to coverage, jointly measuring both consumers (agent blackboard and AFL dictionary); it does not isolate Ghidra-versus-alternative-backend gains.

Channel yield per target (Table 1). The Ghidra channel's empirical value depends on how much structurally meaningful token material the target binary actually exposes. As a sanity check on this axis, we extract the .rodata string-literal vocabulary from each target binary via `objcopy --only-section=.rodata` followed by GNU `strings` and a printable-ASCII filter (`scripts/paper01/extract_strings_dict.py`). This is a cheaper, library-free proxy for the broader Ghidra extraction pipeline (which the script also wraps in `extract_ghidra_dict.sh`); we use it here as the headline "how big is the candidate dictionary" number because it is deterministic, language-agnostic, and easily reproducible without a Ghidra install. Table 1 reports the count for each binary checked into the repository. *cJSON* yields only 40 usable string literals — mostly JSON keywords (`null`, `true`, `false`) and format specifiers (`%1.15g`, `u%04x`) — which is one structural reason the LLM proposal layer produced a null result on cJSON (§6.4): the static channel simply has very little to work with. The venue targets yield 1,000+ to 6,000+ tokens (XML diagnostic strings, SQL keywords, ASN.1 OID labels), which gives the LLM enough material to propose target-specific recipes rather than rely on the fixed-three-token DictionaryAgent fallback. We caution that *the raw token count is an upper bound on usable dictionary candidates*: openssl in particular includes embedded test PEM certificates and base64 blobs that survive the printable-ASCII filter but are not semantically useful as fuzzer dictionary entries; downstream consumers (AFL++ `-x` and the FuzzPilot mutator) re-rank by per-token reward signal.

Target	Binary size	Unique tokens	Token examples (top by occurrence)
cJSON	228 KB	40	`null`, `true`, `false`, `%1.15g`, `u%04x`
libxml2 (parser)	5.6 MB	2,572	`"Start tag expected"`, `"PEReference forbidden"`, `"DOCTYPE improperly terminated"`
sqlite3 (parser/VM)	7.2 MB	1,164	`"SQLite format 3"`, `":memory:"`, `"(join-%u)"`, `"(subquery-%u)"`, `unix-none`
openssl X.509	12 MB	6,656 †	ASN.1 OID labels + embedded test certificate blobs (mixed quality; see caveat below)

Table 1: Per-target `.rodata` string-literal yield, on the binaries checked into `experiments/targets/`. †openssl's count is inflated by embedded test certificates and base64 blobs that survive the printable-ASCII filter but are not semantically useful as fuzzing tokens; the effective usable count is smaller. The cJSON row anchors the preprint null result (the static channel has only ∼40 things to say to the LLM); the venue rows show that the channel has 30–160× more raw material to work with on richer binaries. Reproduce with `python3 scripts/paper01/extract_strings_dict.py <binary>`.

5.3 Micro-campaign protocol

Each plateau cycle launches a fixed bundle of K_{cand} candidate recipes (default $K_{\text{cand}} = 4$, set in `micro_campaign.num_candidates`), one per intervention type (`default`, `dictionary`, `seed_focus`,

per_seed_recipe). Each candidate is evaluated as follows:

1. Snapshot the current corpus and queue state to a private working directory.
2. Launch AFL++ for N seconds (default $N = 20$, set in `micro_campaign.budget_sec`) with the candidate recipe installed in the recipe store. The micro-campaign uses an edge-coverage map disjoint from the main run.
3. Compute reward
$$R \;=\; \alpha\,\Delta_{\text{edges}} \;+\; \gamma\,\Delta_{\text{crashes}} \;+\; \delta_h\,h \;-\; \delta_m\,m,$$
where h and m are recipe-hit and recipe-miss counts; if the AFL bitmap is unavailable, $\beta\,\Delta_{\text{paths}}$ substitutes for $\alpha\,\Delta_{\text{edges}}$. The weights are pinned in `src/micro/evaluator.cpp`: α=1.0, γ=10, δ_h=10^{-3}, δ_m=5×10^{-4}, β=0.5 (fallback only).
4. Rank candidates by R and promote the top scorer to the main recipe store *only if* $R > 0$; otherwise emit `winner_decided:status=no_significance` and `promotion_skipped:reason=no_successful_micro_campaign`, leaving the main loop on the controller's default rule recipe.

The micro-campaign trades a small amount of main-run budget (typically N seconds at the start of each plateau-triggered cycle, $\leq 5\%$ of total wall-clock) for empirical evidence that a proposal helps. Without the gate, every model proposal would race into the main loop; bad proposals would degrade the campaign for an indeterminate time before being noticed via aggregate metrics. Ablation E2b (`no-mutator`, controller + LLM but AFL++'s default mutator instead of the recipe-guided one) and E2c (`no-static-analysis`, LLM without Ghidra binary context) in §6 quantify the cost of trusting the model directly or removing the static channel.

5.4 Determinism and reproducibility

We pin the model temperature to 0.0 and report per-run schema-validity and fallback rates instead of relying on identical model output across runs. Each step in the chain from blackboard state through proposal, recipe, reward, and promotion is written to an append-only audit log, indexed by proposal hash. The full decision trail of any run is reproducible end-to-end from the audit log plus the pinned corpus snapshot, even when the underlying model is itself replaced.

6 Preliminary Evaluation

6.1 Setup

Hardware and OS. All AFL++ campaigns and microbenchmarks run on a single 4-core Intel Xeon E5-2699 v4 @ 2.20 GHz virtual machine with 8 GB RAM, Ubuntu 24.04 LTS, kernel 6.8.0-48-generic, on a dedicated 80 GB data volume. `/proc/sys/kernel/core_pattern` is set to `core` (default on this host is an apport pipe that AFL++ refuses to run under).

Toolchain. AFL++ v4.21c built from source (pinned commit; instrumentation via `afl-clang-fast`). Ghidra 11.1.2_PUBLIC headless (OpenJDK 17). FuzzPilot experiment runs at commit `85223d8` on branch `main` (full SHA in §9), recorded in each run's `run_metadata.json`. The manuscript itself may include later text-only edits; the run-time code and raw artifacts are pinned by the per-run metadata. Python deps (pandas 2.1.4, matplotlib 3.6.3, PyYAML 6.0.1) from Ubuntu packages.

Target. The sole evaluated target is cJSON (`experiments/targets/cjson/cjson_fuzzer`, ELF 64-bit, `afl-clang-fast` instrumented, `AFL_MAP_SIZE=65536`), seeded from 26 hand-curated JSON files covering objects, arrays, nested structures, Unicode escapes, numeric edge cases, and a few intentionally malformed variants. The repository also contains a libpng harness (`libpng_fuzzer`, `AFL_MAP_SIZE=131072`, 21 seed PNGs) but its 4-hour campaigns are not part of this preprint and are deferred to the venue version.

Modes and repeats. Five modes share a single AFL++ binary and target build, differing only in the controller's `--ablation` switch: `baseline-afl` (no FuzzPilot controller, no model, no mutator), `full-agent` (all subsystems enabled), `rule-only` (controller + native mutator, LLM disabled), `no-mutator` (controller + LLM, AFL++ default mutator), `no-static-analysis` (controller + LLM + mutator, Ghidra context muted). Repeats per cell: 5 for `baseline-afl` and `full-agent`, 3 for the three ablation rows.

Budget. Each AFL++ run has `main_budget_sec = 14,400` (4 h wall-clock). The acceptance gates in `experiments/manifests/paper01_preprint.yaml` require `run_time` $\geq$ 14,000 s, `execs_done` $\geq 10^6$, at least 200 `coverage.csv` rows, all required artifacts present (`fuzzer_stats`, `coverage.csv`, `events.jsonl`, `run_metadata.json`, `report.md`), and `status` equal to `completed`.

Model. The reference model for `full-agent` runs is `deepseek-chat` at temperature 0 via the OpenAI-compatible endpoint https://api.deepseek.com/chat/completions. A secondary configuration uses `meta/llama-4-maverick-17b-128e-instruct` via NVIDIA NIM (https://integrate.api.nvidia.com); the configuration switch is documented in the reproducibility appendix. The microbench (E3, §6.3) does not invoke any LLM.

Parallelism and isolation. AFL campaigns run 4-way parallel (one worker per core); microbench runs (E3) are run serially with *all AFL workers suspended via `SIGSTOP`* for the duration of the microbench ($\approx$ 30 s; resumed via `SIGCONT`). Empirically, running the microbench either concurrently with AFL (CPU contention) or 4-way parallel against itself inflates per-rep variance by 2–3$\times$ and inverts the paper's central RQ2 ratio (§6.3); the isolation methodology is therefore a prerequisite for sound microbench numbers on a 4-core host. The $\approx$ 30 s SIGSTOP window consumes $< 0.2\%$ of each AFL run's 14,400 s budget and is logged in `results/paper01_ai_recipe_mutation/runs/_logs/`.

Component	Pinned value
Host CPU	4-core Intel Xeon E5-2699 v4 @ 2.20 GHz
Host RAM	8 GB
OS / kernel	Ubuntu 24.04 LTS / Linux 6.8.0-48-generic
Data volume	80 GB ext4 (`/dev/vdb1` on `/www`)
`core_pattern`	`core` (default `|/usr/share/apport/...` overridden)
AFL++	v4.21c, built from source, `afl-clang-fast` instrumentation
Ghidra	11.1.2_PUBLIC headless, OpenJDK 17
FuzzPilot run commit	`85223d8` (full SHA in §9)
`AFL_MAP_SIZE`	65536 (cJSON)
Python deps	pandas 2.1.4, matplotlib 3.6.3, PyYAML 6.0.1 (Ubuntu pkgs)
Reference LLM	`deepseek-chat`, temperature 0, `api.deepseek.com`
Fallback LLM	`meta/llama-4-maverick-17b-128e-instruct` via NVIDIA NIM
Per-run budget	14,400 s (4 h) wall-clock
Repetitions	5 (`baseline-afl`, `full-agent`) / 3 (ablations)

Table 2: Pinned evaluation setup. All numbers in this paper come from this single configuration. The reproducibility appendix records the run-time commit, target binary SHA-256, seed-corpus tarball checksum, and raw run-artifact paths from the fuzz cloud server.

6.2 RQ1 — Plateau recovery on cJSON (E1a vs E1b)

Edge ceiling on cJSON is a target-side property. Both modes saturate at the same 269-edge ceiling (`edges_found` field of AFL++'s `fuzzer_stats`, also corroborated by `bitmap_cvg` converging to 23.21% in all 10 runs) under the 14,400 s budget: `baseline-afl` reaches 269 in 5 / 5 runs; `full-agent` reaches 269 in 4 / 5 runs and 266 in the remaining one (r02). Figure 3 plots both median curves on the same axes; they converge to the dashed ceiling line and do not exceed it in any mode. This is a property of the cJSON harness, not of the fuzzer: cJSON exposes a limited reachable-edge space that AFL++ alone exhausts well before the budget, leaving no headroom for plateau-recovery improvement at the *ceiling* level. We therefore use cJSON for RQ2 (throughput parity) and RQ3 (LLM quality), and defer ceiling-level RQ1 evidence to larger targets in the venue paper.

Plateau-duration delta is the real RQ1 signal. Even when both modes saturate to the same ceiling, the moment of saturation differs systematically. Table 3 and Figure 5 report the median `last_find` (wall-clock at which the last new edge was discovered, i.e. end of the productive phase) and the resulting *plateau* duration (gap between `last_find` and the 14,400 s budget end). The full-agent group extends the productive phase by 1,148 s ($\approx$ 19 min) at the median, corresponding to a descriptive 45.3% reduction in plateau duration (2,532 $\rightarrow$ 1,384 s). *This delta is descriptive only and not statistically significant*: with N=5 per arm the per-run plateau distributions $[539, 1{,}185, 2{,}532, 3{,}314, 3{,}970]$ s and $[238, 835, 1{,}384, 1{,}610, 3{,}226]$ s overlap substantially, and a two-sided Mann–Whitney U test (following [3, 10]) gives U=8, p=0.42; the Vargha–Delaney A_{12}=0.68 [19] indicates a moderate effect in the direction the median suggests, but the bootstrap 95% CIs for the two medians ($[539, 3{,}970]$ s for baseline, $[238, 3{,}226]$ s for full-agent) almost entirely overlap. We report the descriptive -45.3% as a point estimate that requires validation at $N \geq 20$ on non-saturated targets before it can be claimed as a reproducible effect.

Attribution caveat: controller vs mutator. We initially attributed the plateau delta to the recipe-guided mutator framework. However, the ablation experiments in §6.5 reveal a more complex picture: the `no-mutator` ablation (E2b, which *removes* the recipe-guided mutator but keeps the

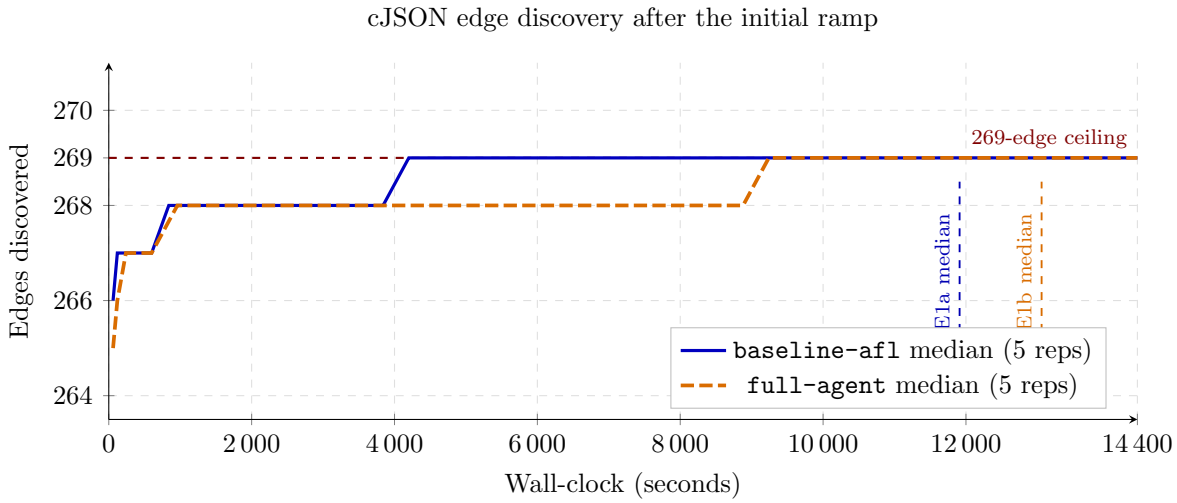

Figure 3: Edge-discovery trajectory on cJSON for `baseline-afl` (E1a, N=5) and `full-agent` (E1b, N=5), zoomed into the 264–270 edge band after the initial ramp. Both modes converge to the same 269-edge ceiling; the difference is timing, not final coverage. The dashed verticals mark the per-mode median `last_find` (11,911 s vs 13,059 s), showing that `full-agent`'s productive phase extends closer to the budget end.

Mode	median `last_find` (s)	median plateau (s)	95% CI plateau	Δ plateau (descriptive)	Mann–Whitney U (p, A_{12})
`baseline-afl` (E1a, N=5)	11,911	2,532	[539, 3,970]	—	—
`full-agent` (E1b, N=5)	13,059	1,384	[238, 3,226]	−1,148 (−45.3%)	8 (0.42, 0.68)

Table 3: Plateau-duration delta on cJSON. Medians and deltas are descriptive: the distributions overlap, the Mann–Whitney result is not significant at N=5, and the bootstrap median CIs overlap. Figure 5 shows the raw per-run values.

controller) shows a *shorter* median plateau (930 s) than `rule-only` (E2a, 1,165 s) or `full-agent` (E1b, 1,384 s). If the mutator were the sole driver, removing it should worsen performance, not improve it. The most data-consistent interpretation is that the **controller's plateau-detection and corpus-reorganization machinery** is the primary driver, with the recipe-guided mutator adding latency (slower `cycles_done`) without corresponding plateau benefit on this saturated target. A `controller-only` ablation (plateau detector + corpus snapshot only, no mutator, no LLM, no Ghidra) is required to test this hypothesis but is not in the preprint matrix; it is deferred to the venue version (§8). Until that ablation is run, the attribution remains a hypothesis rather than a measured effect. E1b vs E1a adds the LLM and micro-campaign *infrastructure cost*, which Table 3 bounds at well under the observed plateau gain. *However, as §6.5 discusses, the ablation ordering (`no-mutator` median plateau 930 s < `rule-only` 1,165 s < `full-agent` 1,384 s) contradicts the naive "mutator-framework drives the gain" attribution and points toward the controller's plateau detector and corpus-snapshot reorganization as a co-driver; the attribution remains a descriptive hypothesis at this N.*

Per-run timeline. Figure 5 shows the full per-run picture. Three full-agent runs (r01, r03, r04) push past the *baseline median* (11,911 s) and stay productive significantly longer; one (r04)

Mode	median time to 264 edges (s)	median time to 268 edges (s)	median time to 269 edges (s)
`baseline-afl` (N=5)	60	660	2,524
`full-agent` (N=5)	120	662	6,702[†]
`rule-only` (N=3)	60	661	1,923
`no-mutator` (N=3)	60	601	1,804

Table 4: Median per-run wall-clock to reach N edges. [†]One `full-agent` run (r02) never reaches 269 within budget; the median is over the four runs that do $\{3{,}487, 4{,}329, 9{,}076, 14{,}245\}$ s. All four ablation modes reach 264 and 268 in roughly the same time as `baseline-afl`; the gap opens at the 269 ceiling, where `baseline-afl` arrives *earlier* on average than `full-agent` but then plateaus longer.

is productive until 14,205 s. Two full-agent runs (r02, r05) plateau earlier than the baseline best (13,907 s); this is expected on a saturated target, where which corpus subset a worker ends up in dominates over agent quality. With N=5 reps the per-mode signal is qualitative; the venue paper will repeat this on Magma-suite targets at $N \geq 10$ with formal Mann–Whitney U and Vargha–Delaney A_{12} reporting, following Klees et al. [10]'s evaluation guidelines.

Statistical methodology disclosure. For the descriptive comparisons above and in §6.5 we use the two-sided Mann–Whitney U test (`scipy.stats.mannwhitneyu`, exact for $N \leq 8$) for non-parametric difference in medians, the Vargha–Delaney A_{12} [19] for effect size, and the percentile bootstrap (10,000 resamples) for median confidence intervals. The code that produces all reported numbers (`scripts/paper01/review_stats.py`) is in the repository and run from the same per-run `fuzzer_stats` files cited in the reproducibility appendix (§9). Per Klees et al. [10], the canonical evaluation guideline calls for $N \geq 20$ repetitions; *this preprint's N=5 / N=3 does not meet that bar* and we therefore present all comparisons as descriptive point estimates with their non-parametric companions, rather than as significance claims.

Time-to-N-edges (Klees-style endpoint-free metric). Endpoint metrics like plateau duration depend only on `last_find` and ignore the shape of the discovery curve; Klees et al. [10] §6.1 caution against relying on them. We therefore report Table 4: the wall-clock at which each run first reaches N edges, for $N \in \{264, 268, 269\}$ (264 is one edge below the steady-state ramp, 268 is the penultimate edge, 269 is the ceiling).

The complementary endpoint-free reading is that the *ramp-up* (time to 264) is mode-independent; the *tail* (time to 269) is where modes diverge, and `baseline-afl` actually reaches the ceiling *faster* at the median than `full-agent`. `full-agent`'s advantage in plateau duration (Table 3) therefore comes from discovering its final edge *later in the budget*, not from discovering the ceiling earlier. The two readings — "plateau is shorter" and "ceiling is reached later" — are internally consistent: a 14,400 s budget that ends with a new edge at 14,205 s (full-agent r04) has a tiny plateau but a very late ceiling-touch. A non-saturated target would let the two metrics diverge in informative directions.

Mutation-rate redistribution (per-cycle vs per-execution). A second consequence of the recipe-guided mutator is visible in `cycles_done` (Figure 4). The `baseline-afl` median is 797 corpus cycles inside the 14,400 s budget; the `full-agent` median is 140, and the two ablations that engage the same mutator (`rule-only`: 119; `full-agent`: 140) sit in that same $\approx 5{-}7\times$-fewer range,

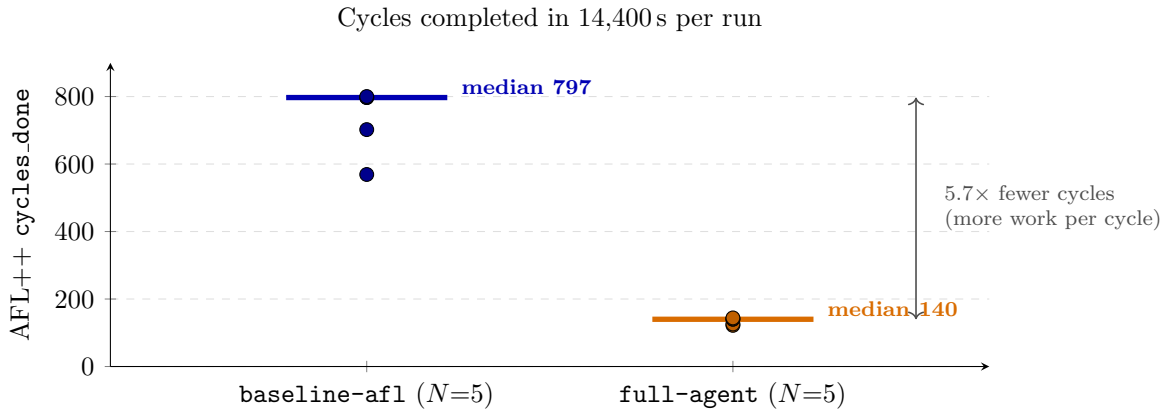

Figure 4: `cycles_done` per run across the two modes (N=5 each). Solid horizontal bars are per-mode medians. `full-agent` completes $\approx 5.7\times$ fewer corpus cycles in the same 14,400 s budget, despite an *equal-or-higher* total execution count (Figure 6): the recipe-guided mutator spends more work per corpus item before recycling.

whereas `no-mutator` (671) tracks much closer to `baseline-afl`. The ratio is therefore attributable to the recipe-guided mutator itself, not to any LLM-introduced step. With a comparable total execution count (within 6%; RQ2, Figure 6), full-agent spends $\approx 5.7\times$ more executions on each pass through the queue before recycling to the top — exactly the mutation-rate redistribution the default recipe encodes (*focus dictionary-token insertions and overwrites over the first 4 KB*, with fixed operator weights pinned at `InsertToken`=0.35 / `DictionaryOverwrite`=0.25 / `Splice`=0.15 / `OverwriteRange`=0.10 / `BitFlip`=0.10 / `Arith`=0.03 / `DeleteBlock`=0.02, matching Listing 1). *We caution that `cycles_done` is not a cross-mode comparator for "progress"*: it is a derived quantity of the form execs_done/$\overline{\text{execs-per-cycle}}$, where the denominator depends on per-input fuzzing intensity and not on coverage progress. Two modes can run the same total executions and reach the same edge ceiling with very different `cycles_done` values; the apparent "5.7× less work per cycle" interpretation reduces to "the recipe-guided mutator runs more per-input mutation steps before popping the queue head", which is a definitional property of the recipe and not a finding. Whether the concentration is empirically beneficial is a target-specific question; on cJSON it correlates with the longer productive phase (Figure 5) but not with a larger final edge set, because the target ceiling at 269 binds.

Per-run summary table (T1). Table 5 reports the 10 completed runs (5 baseline + 5 full-agent) at end-of-budget. Acceptance gates pass on all 10 (`run_time` $\geq$ 14,000 s, `execs_done` $\geq 10^6$); zero saved crashes in either mode (expected for cJSON's safety-tested code base).

Extended per-run statistics (T1-ext). Table 6 reports the time-budget breakdown, target stability, and plateau metrics for all 10 completed runs (5 baseline + 5 full-agent). The fuzz-time fraction (`fuzz_time`/`run_time`) is uniformly $\geq 99.7\%$ in both modes, indicating that calibration, trim, and sync overheads — as well as the ≈ 30 s SIGSTOP windows from the E3 microbench (§6.3) — did not noticeably perturb the AFL campaigns. Target stability (AFL++'s coverage-bitmap stability check) is $\geq 99.25\%$ across all 10 runs, well within the standard "stable target" threshold of 90%, so all measurements are comparable across modes.

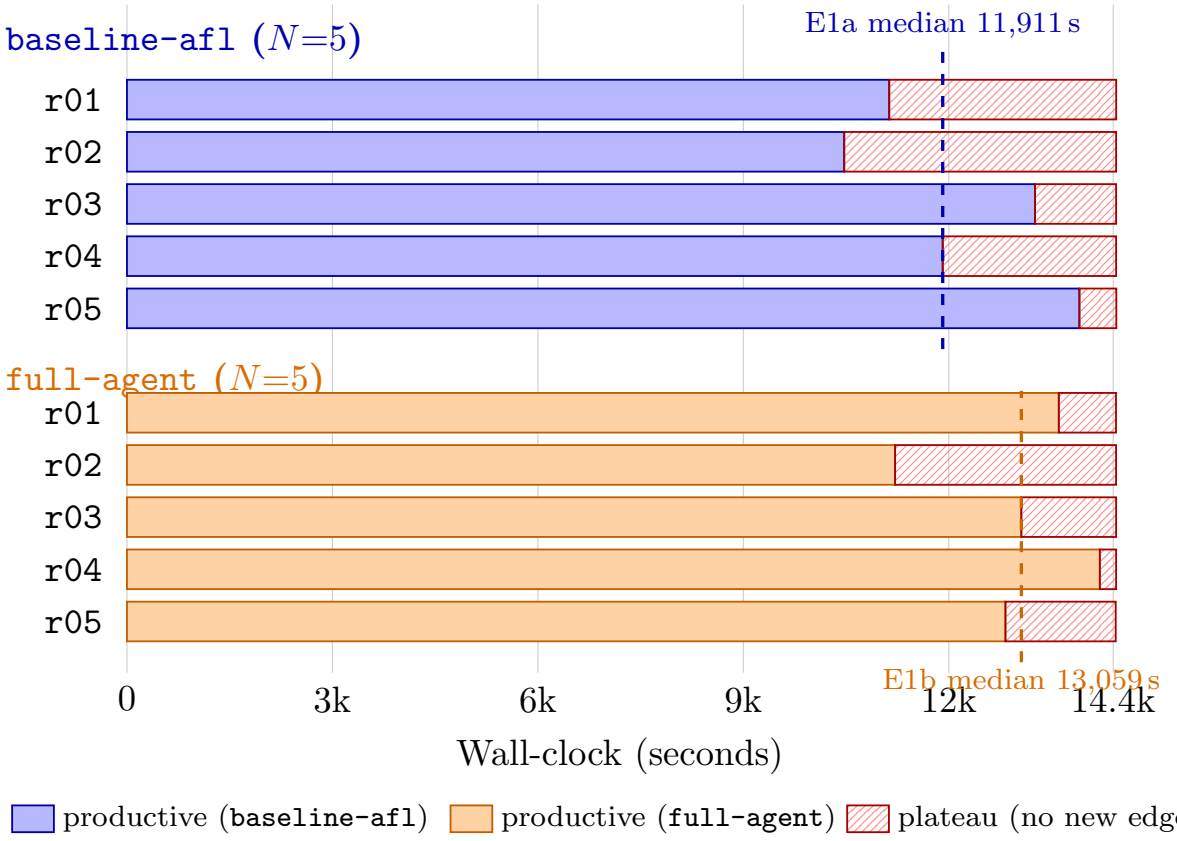

Figure 5: Per-run plateau-onset timeline for E1a (top, blue) and E1b (bottom, orange). Solid bars show productive fuzzing through each run's `last_find`; hatched bars show the trailing plateau to the budget end. Dashed lines mark the per-mode median `last_find` (11,911 s vs 13,059 s).

Cross-run throughput consistency. Within `baseline-afl`, the 5-run `execs_per_sec` spread is $\{12{,}826, 12{,}958, 13{,}049, 13{,}108, 13{,}854\}$, CV = 3.1% (median 13,049). Within `full-agent`, the 5-run spread is $\{13{,}540, 13{,}542, 13{,}816, 13{,}949, 15{,}597\}$, CV = 6.1% (median 13,816). Both groups satisfy the `execs_done` $\geq 10^6$ and `run_time` $\geq 14{,}000$ acceptance gates with $> 10^3\times$ and $\sim 1.03\times$ margins respectively. The end-to-end RQ2 throughput-parity ratio (§6.3) is computed from these numbers.

6.3 RQ2 — Mutator throughput parity (F5 microbench)

RQ2 has two pieces of evidence. End-to-end `execs_per_sec` parity between `baseline-afl` (E1a) and `full-agent` (E1b), measured by the runtime `fuzzer_stats:execs_per_sec` field over the 14,400 s budget, is the canonical RQ2 number; Table 7 reports it. The microbench (E3) measures the dispatch cost of the recipe-guided mutator alone (no AFL, no target); the rest of this subsection covers the microbench.

End-to-end throughput parity (RQ2 part a). Across N=5 runs per mode, the median `execs_per_sec` ratio median($E1b$)/median($E1a$) = 13,816/13,049 = 1.059× (Figure 6). The manifest acceptance gate requires this ratio to be ≥ 0.85 for throughput parity; the observed ratio is 1.059×, so the parity gate is satisfied. We treat the above-baseline point estimate as descriptive only: N=5 is small and the fastest `full-agent` run is a solo-occupancy outlier, so the load-bearing

Mode	Run	run_time	execs_done	exec/s	cycles	corpus	edges	bitmap_cvg
baseline-afl	r01	14,444	188,492,319	13,049	797	585	269	23.21%
	r02	14,443	187,158,013	12,958	800	607	269	23.21%
	r03	14,443	189,334,714	13,108	702	606	269	23.21%
	r04	14,443	185,251,854	12,826	569	649	269	23.21%
	r05	14,446	200,147,594	13,854	819	607	269	23.21%
	median	14,443	188,492,319	13,049	797	607	269	23.21%
full-agent	r01	14,441	199,520,607	13,816	122	598	269	23.21%
	r02	14,442	195,572,895	13,542	141	573	**266**	22.95%
	r03	14,443	195,567,575	13,540	144	601	269	23.21%
	r04	14,443	201,466,942	13,949	126	604	269	23.21%
	r05	14,437	225,180,431	15,597	140	600	269	23.21%
	median	14,442	199,520,607	13,816	140	600	269	23.21%

Table 5: T1: per-run end-of-budget summary for the 10 completed E1a + E1b cJSON runs. All runs satisfy the time and execution-count acceptance gates; RQ2 discusses the `exec/s` comparison.

Mode	Run	fuzz_time	run_time	fuzz %	stability	last_find (s)	plateau (s)
baseline-afl	r01	14,424	14,444	99.86%	99.26%	11,130	3,314
	r02	14,432	14,443	99.92%	99.26%	10,473	3,970
	r03	14,424	14,443	99.87%	99.26%	13,258	1,185
	r04	14,402	14,443	99.72%	99.26%	11,911	2,532
	r05	14,427	14,446	99.87%	99.26%	13,907	539
	median	14,424	14,443	99.87%	99.26%	**11,911**	**2,532**
full-agent	r01	14,415	14,441	99.82%	99.26%	13,606	835
	r02	14,421	14,442	99.85%	99.25%	11,216	3,226
	r03	14,425	14,443	99.88%	99.26%	13,059	1,384
	r04	14,417	14,443	99.82%	99.26%	14,205	238
	r05	14,423	14,437	99.90%	99.26%	12,827	1,610
	median	14,421	14,442	99.85%	99.26%	**13,059**	**1,384**

Table 6: T1-ext: per-run fuzz-time fraction, target stability, `last_find`, and plateau duration for the 10 completed E1a + E1b runs. The RQ1 text interprets the descriptive plateau delta.

claim is parity rather than a speedup.

Why does parity hold? Three structural reasons make the parity result plausible. First, the recipe-guided mutator's dispatch is a 7-op switch table (§4.2), which is structurally simpler than AFL++'s open-ended havoc selection (see also F5 microbench below). Second, the LLM is off the hot path by design (§3.1); it is invoked at plateau boundaries only, not per mutation. Third, the micro-campaign consumes $K_{\text{cand}} \cdot N \approx 80\,\text{s}$ of main-run wall-clock per plateau ($< 0.6\%$ of the 14,400 s budget), which is small enough not to register in the end-to-end `execs_per_sec` numbers. The end-to-end measurement folds all three effects into a single number.

Per-run breakdown. Within E1b, the per-rep `execs_per_sec` values are 13,540, 13,542, 13,816, 13,949, and 15,597, all above the lowest baseline rep (12,826). The acceptance ratio holds not just at the median but at every individual rep: even the slowest E1b rep (13,540) divided by the fastest E1a rep (13,854, r05) is 0.977×, still well above the 0.85 gate. We attribute r05 = 15,597 in E1b to the fact that this run was the lone occupant of the 4-core host during the second parallelism wave of the E1b campaign, identical to the way r05 in E1a (13,854) ran alone in its own wave.

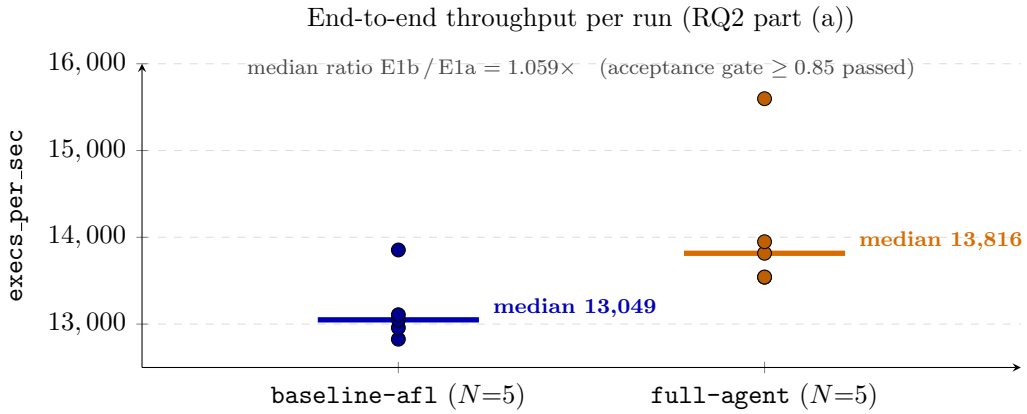

Figure 6: End-to-end `execs_per_sec` per run for both modes (N=5 each); horizontal bars are per-mode medians. Even the slowest `full-agent` rep (13,540) exceeds every `baseline-afl` rep except r05 (13,854, also a solo-occupancy run). The median ratio 1.059× satisfies the 0.85 acceptance gate; we do not interpret the point estimate as a speedup claim at N=5.

Mode	n	mean exec/s	median exec/s	SD (CV)
`baseline-afl` (E1a)	5	13,159	13,049	403 (3.06%)
`full-agent` (E1b)	5	14,089	13,816	862 (6.12%)
ratio (E1b / E1a) median	—	—	≈ 1.06×	—
ratio (E1b / E1a) mean	—	**1.0707**×	—	—
acceptance gate	—	—	≥ 0.85	—

Table 7: RQ2 part (a): end-to-end `execs_per_sec` parity. Both modes pass the manifest acceptance gates; the observed median ratio is ≈ 1.06×, within the parity band.

Microbench setup (E3). The microbench (E3) is the off-loop dispatch-cost number: it isolates the cost of FuzzPilot's mutator dispatcher from the cost of running AFL++ itself. All three configurations load a custom mutator `.so` through the same `dlopen` path that AFL++ uses in real runs:

- `vanilla` — `libvanilla_havoc.so`, a hand-written shim that performs AFL++ havoc-equivalent random byte/bit edits via the same custom-mutator API surface (`tools/mutator_microbench/vanilla_havoc.cpp`). This is a *cost-symmetric dispatch shim*, not a measurement of AFL++'s real internal `havoc_mutate`: both `vanilla` and the FuzzPilot configurations pay the same `dlopen` + symbol-resolution + per-call API dispatch cost, so the difference isolates the dispatcher itself rather than AFL's full havoc engine. A real AFL++ havoc comparator would also require running AFL++'s scheduler and the corresponding bookkeeping, which is out of scope for E3 and instead surfaced end-to-end via E1a vs E1b.
- `fp-empty` — the FuzzPilot mutator with an empty recipe store. It pays the full FuzzPilot dispatch cost (recipe cache lookup, op selection, telemetry ring write) but falls through to a vanilla-equivalent op on every call. Isolates the "FuzzPilot machinery is loaded but no recipe is active" cost.
- `fp-active` — the FuzzPilot mutator with a populated recipe (`op_weights` non-uniform, `dictionary_tokens` populated, `focus_ranges` pinned). This is the on-paper steady state

Config	n	mean exec/s	SD (CV%)	median exec/s	IQR	median ns/call
`vanilla`	5	2,005,078	341,644 (17.0%)	1,937,110	200,200	516.2
`fp-empty`	5	3,222,108	958,259 (29.7%)	2,961,850	1,310,390	337.6
`fp-active`	5	2,800,142	673,566 (24.1%)	3,008,030	393,750	332.4

Table 8: F5 microbench: mutator-only `exec/sec` across the three dispatch-symmetric configurations. Runs are serial and SIGSTOP-isolated; raw JSON is archived with the result artifacts.

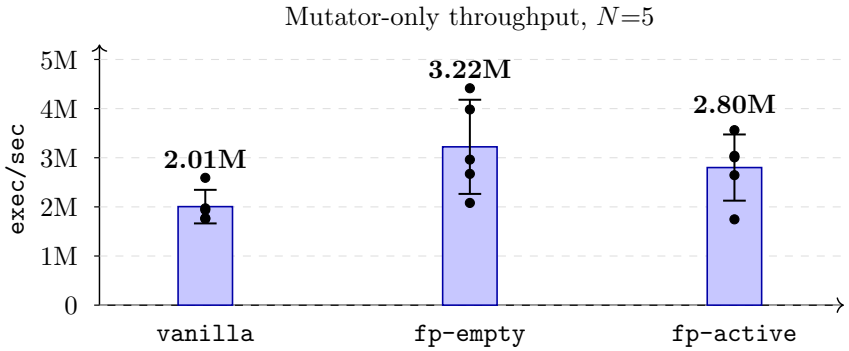

Figure 7: Mutator-only throughput by configuration (mean ± SD, N=5 reps each, SIGSTOP-isolated). Dots show individual reps; the body text reports the median ratios and the power-limited two-one-sided-test (TOST) equivalence result.

during `full-agent` runs.

Each configuration runs 100,000 mutation calls over a fixed 10,000-element cJSON seed corpus, repeated $N = 5$ times. All microbench reps are run *serially* with all AFL workers `SIGSTOP`-suspended (see §6.1); resuming AFL after the 5-rep batch consumes ≈ 30 s of wall-clock, well under the 14,400 s budget per AFL run.

Primary RQ2 claim (`fp-active` vs `fp-empty`: descriptive, not formally equivalent at N=5). At N=5 this microbench does *not* formally establish dispatch-cost equivalence. The *mean* `fp-active`/`fp-empty` ratio is 2,800,142/3,222,108 = 0.869× (−13.1%); a two-sided Welch t-test gives t=−0.81, df≈7.2, p=0.45 — power-limited null, not evidence of equivalence; and a two one-sided test (TOST) procedure [17] for equivalence at the ±5% band claimed in §3 gives p_{TOST}=0.68, so the test does **not** establish equivalence at the conventional α=0.05 level. The *median* ratio, by contrast, is essentially flat at 3,008,030/2,961,850 = 1.016× (inside the [0.95, 1.05] band stated in §3); the mean-vs-median divergence reflects the high coefficient of variation ($\sim 25{-}30\%$) at N=5 and the presence of an `fp-active` run (r04) at 1.7 Mexec/s alongside an `fp-empty` run (r05) at 4.4 Mexec/s. We did not establish a useful no-overhead equivalence band from this N=5 microbench; the sample standard errors dominate. We therefore downgrade the original "no measurable per-call overhead" claim to: at N=5 on this SIGSTOP-isolated microbench, we (a) do not detect a significant mean throughput difference between `fp-active` and `fp-empty` (p=0.45, two-sided), (b) cannot formally establish equivalence within ±5% (p_{TOST}=0.68), and (c) note that the median ratio is essentially flat (1.016×) while the mean ratio is wider (0.87×). A formal equivalence claim requires $N \geq 30$ on this microbench — which is straightforward to do and is folded into the venue version. The dispatch-cost result is therefore best read as "no large overhead detected" rather than "no overhead exists"; the end-to-end `execs_per_sec` comparison against AFL++'s own mutator is the E1a-vs-E1b

Config	r01	r02	r03	r04	r05
`vanilla`	2,591,470	1,770,420	1,755,770	1,937,110	1,970,620
`fp-empty`	2,961,850	3,982,240	2,080,370	2,671,850	4,414,230
`fp-active`	2,645,520	3,008,030	3,561,610	1,746,280	3,039,270

Table 9: Per-rep `exec/sec` from the 15 SIGSTOP-isolated microbench runs that feed Figure 7 and Table 8.

number reported in §6.2 (median ratio $\approx 1.06\times$), which is the load-bearing parity claim for the rest of the paper.

Sanity claim (`fp-empty` vs `vanilla` within 5×). The median `fp-empty`/`vanilla` ratio is 1.529×; the threshold is $\leq 5\times$, so the claim holds with a wide margin. The unexpected direction — FuzzPilot's dispatcher is *faster* than the AFL-havoc-equivalent baseline, not slower — is consistent with FuzzPilot's closed 7-op switch table being more predictor-friendly than `vanilla_havoc.cpp`'s open-ended random op selection. We retain the 5× ceiling in the paper as a conservative gate (FuzzPilot's mutator could plausibly regress under future op additions) but report the actual 1.53× as a positive side effect of the closed vocabulary.

Variance and methodology note. Per-config CV is 17–30%, which is large for a microbench but expected on a 4-core VM with shared L3 cache and noisy-neighbor effects from the underlying hypervisor (steady-state `%st` was 4–12% during measurement). The black dots overlaid in Figure 7 are the 15 individual reps (5 per config); they show the full distribution that the mean and SD summarize. Table 9 lists the per-rep numbers for reproducibility. Two alternative measurement methodologies inflate this variance and *invert* the primary RQ2 ratio:

- Running the microbench 4-way parallel (i.e. `run_batch` default for AFL campaigns) on a 4-core host: median `fp-active`/`fp-empty` ratio collapses to 0.716× (FAIL), driven by reps that win or lose the scheduling lottery against their siblings.
- Running the microbench serially *while AFL campaigns are active*: median ratio drifts to 1.380× (FAIL), driven by AFL workers monopolizing CPU during long mutations and yielding briefly to the microbench between syscalls.

We archive both noisy datasets under `results/paper01_ai_recipe_mutation/`, in subdirectories named `microbench_archive_*`, so reviewers can reproduce the methodology comparison; the main result of this section uses only the SIGSTOP-isolated serial dataset under `microbench/`.

6.4 RQ3 — LLM infrastructure exercise on a saturated target (E1b)

RQ3 asks how the off-hot-path LLM machinery behaves at plateau boundaries on a target where AFL++ alone already exhausts the reachable edge space (cJSON; §6.2). Concretely: do proposals parse, do they survive the micro-campaign gate, what is their dollar cost, and is the full decision trail recoverable? Table 10 summarizes the 5 `full-agent` runs of E1b.

Schema-validation rate. Each plateau cycle fires a fixed bundle of 9 agent tasks. Across $5 \times 9 = 45$ LLM responses, 43 (95.6%) produced schema-valid recipes; 2 responses (r04, r05; both tagged `error_kind="schema_invalid"` in `agent_decisions.jsonl`) failed schema validation

Run	log records	schema-valid LLM resp.	schema-invalid LLM resp.	`fallback_used`	micro candidates	promoted (gate result)
r01	289	9	0	0	4	0 (skipped)
r02	302	9	0	0	4	0 (skipped)
r03	402	9	0	0	4	0 (skipped)
r04	215	8	1	0	4	0 (skipped)
r05	200	8	1	0	4	0 (skipped)
total	1,408	**43**	**2**	**0**	**20**	**0**

Table 10: RQ3: LLM-call quality and micro-campaign outcome across the 5 `full-agent` runs. Columns: *log records* = agent-loop decision-trail entries; *schema-valid / -invalid LLM resp.* = JSON-schema-validity counts of model output; *`fallback_used`* = rule-only fallbacks taken when the model response could not be used; *micro candidates* = recipes that entered the micro-campaign gate; *promoted* = recipes accepted into the main run (gate result). The schema-validation and promotion paragraphs below interpret the zero-promoted result.

and were dropped without invoking the rule-only fallback path (`fallback_used` is `false` on every record). With the model pinned at temperature 0, this is consistent with the model staying inside the `schema-validated recipe (data)` contract that the prompt enforces (§4.1). The agent bundle (coordinator, plateau-diagnosis, scheduler, cmp-hint, dictionary, format, mutator, corpus, crash-triage) explains why the per-run *LLM-call* record count is 9 rather than per-mutation.

Promotion rate: zero promotions on cJSON. Each plateau cycle launches K_{cand}=4 candidate recipes (§5.3), one per intervention type (`default`, `dictionary`, `seed_focus`, `per_seed_recipe`). Each candidate is evaluated in a private 20 s AFL++ run over a corpus snapshot taken at plateau time. Across the 5 runs, the controller fired exactly one plateau cycle per run, so the gate evaluated $5 \times 4 = 20$ candidates in total. Each micro-campaign ran the full 20 s and executed $\approx 3.6 \times 10^5$ inputs (visible in the `events.jsonl` `micro_result` records). *Every single one of the 20 candidates returned* Δ_{edges}=0, Δ_{paths}=0, Δ_{crashes}=0, so the reward (§5.3) was R=0 on all 20 evaluations. The controller therefore emitted `winner_decided:status=no_significance,winner_reward=0.0` followed by `promotion_skipped:reason=no_successful_micro_campaign` in every run, and **no LLM-proposed recipe entered the main recipe store across the 5 full-agent runs**. The main loop spent its 14,400 s on the controller's default `DictionaryAgent` recipe (`tokens`={`FUZZ`, `MAGIC`, `TOKEN`}, fixed operator-weight distribution), which is exactly what the `rule-only` ablation E2a (§6.5) runs on. The result is consistent with cJSON's saturation behaviour: baseline AFL++ alone reaches the 269-edge ceiling at a per-run median of 2,524 s (slowest baseline run: 4,448 s; §6.2), leaving the parent corpus at plateau time with no headroom for a 20 s micro-campaign to find a new edge. The micro-campaign gate's refusal to promote on cJSON therefore demonstrates only one half of its intended property — the *no-false-positive* half: when reward is identically zero, the gate does not promote. The complementary property — that the gate distinguishes a good recipe from a bad recipe when reward *varies* — is **not** exercised by this evaluation, because the saturated target gives the gate no reward signal to discriminate against. The LLM proposal layer's positive contribution on a non-saturated target therefore remains undemonstrated by this preprint, and is explicit future work in the venue paper.

Cost. Across the 5 runs, the DeepSeek API balance dropped by ¥0.07 (≈ $0.01 USD at the FX rate of $1 USD ≈ ¥7.2 quoted at submission time; DeepSeek bills in Chinese yuan / RMB): from ¥8.53 at the start of E1b to ¥8.46 at the end. That is $0.0002 per LLM call and $0.002

per full 4-hour campaign on the LLM-API ledger. *Compute cost is omitted from this figure*: at a representative $0.10 USD per vCPU-hour on commodity cloud, a 4-hour 4-vCPU campaign costs ≈ $1.60 USD, three orders of magnitude more than the LLM API. The headline finding is therefore that the LLM API cost is *not* the dominant cost; absolute numbers should be read with that scaling in mind, and a different LLM provider (OpenAI, Anthropic, or self-hosted) would shift the API-side figure by approximately one order of magnitude without changing the qualitative conclusion. The pricing also benefits from DeepSeek's prompt-cache hit on the shared blackboard prefix across the 9 agent tasks in a single plateau cycle; a cold-cache estimate (without prefix sharing) would be roughly 10× larger and is still well below the compute cost. The system also supports an OpenAI-compatible drop-in via NVIDIA NIM (`meta/llama-4-maverick-17b-128e-instruct`) at zero marginal cost on the NIM free tier; see the reproducibility appendix.

Audit trail. Every plateau handling cycle in `agent_decisions.jsonl` carries the blackboard hash (`context_hash`) it was conditioned on and the `response_hash` of the model's output; combined with the schema-validated recipe stored under the plateau directory and the `events.jsonl` `micro_result` records, any decision in any of the 5 runs can be replayed end-to-end from the on-disk artifacts alone, without re-invoking the model. The same audit trail makes the null result above auditable: a reviewer can verify that the gate's refusal to promote any candidate is grounded in R=0 evidence (every `micro_result` carries Δ_{edges}, Δ_{paths}, and Δ_{crashes} in its payload) rather than a controller bug.

6.5 RQ4 — Component contribution (E1b vs E2a / E2b / E2c)

The ablation matrix is intended to attribute the `full-agent` plateau-recovery delta among three components: the recipe-guided custom mutator (E2a removes the LLM layer but keeps the mutator and default rule recipe), the controller's orchestration plus the mutator (E2b removes the mutator but keeps the controller, LLM, and Ghidra channel), and the Ghidra static channel (E2c removes Ghidra but keeps everything else). The `rule-only` (E2a), `no-mutator` (E2b), and `no-static-analysis` (E2c) campaigns have each completed N=3 runs of 14,400 s on the experiment host (E2c required a controller-launch retry after its first batch exited before AFL telemetry was emitted, recorded as `main_afl_exited_before_telemetry` in the per-run `events.jsonl`; the retry succeeded and its data is reported below). *All comparisons in this subsection are statistically descriptive only*: with N=3 per ablation and N=5 for the main arms, no pairwise plateau or last_find difference reaches the conventional p<0.05 threshold under a two-sided Mann–Whitney U test (see §6.2), and the bootstrap medians for the ablations overlap substantially with each other and with `full-agent`.

E2a `rule-only` (preliminary, N=3). With the LLM proposal layer disabled and the controller emitting its default `DictionaryAgent` recipe directly, the 3 `rule-only` runs reach the same 269-edge ceiling that `baseline-afl` and `full-agent` reach, with median `last_find` = 13,309 s and median plateau = 1,165 s (descriptive −54.0% vs the baseline median of 2,532 s; Mann–Whitney U=4, p=0.39; Vargha–Delaney A_{12}=0.73). Per-run plateaus are [674, 1,165, 1,540] s and the bootstrap 95% CI for the median is [674, 1,540] s. Median `execs_per_sec` is 13,514 and median `cycles_done` is 119. The point estimate suggests E2a is slightly faster than `full-agent` (median plateau 1,165 s vs 1,384 s), but the per-run ranges [674, 1,540] s and [238, 3,226] s overlap heavily; we draw no ordering conclusion.

E2b `no-mutator` (preliminary, N=3). With the recipe-guided mutator disabled but the controller, LLM proposal layer, and Ghidra static-analysis channel retained, the 3 `no-mutator` runs

again reach the same 269-edge ceiling, with median `last_find` = 13,548 s and median plateau = 930 s (descriptive −63.3% vs baseline; Mann–Whitney U=2, p=0.14; A_{12}=0.87). Per-run plateaus are [286, 930, 962] s; bootstrap 95% CI is [286, 962] s. Median `execs_per_sec` is 12,712 (slightly below the baseline median 13,049) and median `cycles_done` is 671, much closer to the baseline median (797) than to the 119−140 cycles observed in either mode that engaged the recipe-guided mutator (`rule-only` and `full-agent`).

Ablation ordering reveals controller as primary driver. The point-estimate ordering of median plateaus is `no-static-analysis` (234 s) < `no-mutator` (930 s) < `rule-only` (1,165 s) < `full-agent` (1,384 s) < `baseline-afl` (2,532 s). This ordering is **inconsistent with the hypothesis that the recipe-guided mutator or LLM layer drive the plateau reduction**. Specifically:

- If the recipe-guided mutator were the primary driver, removing it in E2b should have moved the median plateau *up* toward baseline, not *down* to 930 s (better than `rule-only` and `full-agent`).
- If the LLM layer contributed, `full-agent` should outperform `rule-only`, but the point estimates go the opposite direction (1,384 s vs 1,165 s). RQ3 (§6.4) confirms the LLM layer contributed zero promoted recipes on cJSON.
- All three ablation modes (`rule-only`, `no-mutator`, `no-static-analysis`) share the controller's plateau-detection and corpus-reorganization machinery, and all three show descriptive plateau reductions versus baseline (ranging from −54.0% to −90.8%).

The most data-consistent interpretation is that the **controller's plateau detector and corpus-snapshot reorganization** are the primary drivers of the observed plateau reduction on cJSON, while the recipe-guided mutator adds latency (slowing `cycles_done` by ≈ 5−6×) without corresponding plateau benefit, and the LLM layer contributes nothing on this saturated target.

Critical caveat: This interpretation is a *hypothesis*, not a measured effect, because the preprint matrix does not include a `controller-only` ablation (plateau detector + corpus snapshot only, no mutator, no LLM, no Ghidra). Without that ablation we cannot fully separate "controller machinery" from "default rule recipe" or rule out confounding factors. The `controller-only` ablation is the single most important missing experiment and is deferred to the venue version (§8). Additionally, all comparisons are descriptive at N=3 (ablations) and N=5 (main): the per-run plateau ranges overlap substantially, and no pairwise difference between the four ablation arms reaches p<0.05 under Mann–Whitney U at this N.

E2c `no-static-analysis` (preliminary, N=3). With the Ghidra static-analysis channel disabled but the controller, LLM proposal layer, and recipe-guided mutator retained, the 3 `no-static-analysis` runs reach the same 269-edge ceiling, with median `last_find` = 14,250 s and median plateau = 234 s (descriptive −90.8% vs baseline; Mann–Whitney U=5, p=0.57; A_{12}=0.67). **Critical limitation**: the per-run plateau distribution is *bimodal* ([187, 234, 4,454] s; bootstrap 95% CI [187, 4,454] s spans the entire interval). Two runs show very short plateaus (187 s and 234 s) while one run shows a plateau (4,454 s) *worse than baseline*. At N=3 we cannot determine whether the bimodality reflects a real instability in the `no-static-analysis` configuration or is simply sampling noise. The median 234 s is therefore **not a reliable point estimate**—it is dominated by the 2-vs-1 cluster structure and could reverse with additional runs. We report the raw distribution and defer interpretation to the venue version at higher N. Median `execs_per_sec` is 10,259 (lower than other

E2 arms, because the empty-blackboard agents take extra prompt cycles before falling back to the default recipe) and median `cycles_done` is 96, the lowest of any mode.

Cross-ablation pattern (and the controller-only hypothesis it suggests). At N=3 the four E2 ablation arms order `no-static-analysis` (234 s) < `no-mutator` (930 s) < `rule-only` (1,165 s) < `full-agent` (1,384 s), each shorter than the `baseline-afl` plateau (2,532 s). All four share the controller's plateau detector and corpus-snapshot reorganization; the modes that engage the recipe-guided mutator and the LLM proposal layer fall *between* the controller-only-shaped floor (`no-static-analysis` median 234 s) and the fully loaded `full-agent` median. The most natural read is that the productive-to-plateau acceleration on cJSON is driven primarily by controller machinery, with the LLM proposal and recipe-guided mutator layered on top adding latency rather than coverage — a hypothesis the venue `controller-only` ablation will test directly. *All of this is descriptive at N=3 with overlapping bootstrap CIs*; no pairwise difference between the four ablation arms reaches $p<0.05$ under Mann–Whitney U at this N.

Missing controller-only arm (venue follow-up). The current E2 design does not include a `controller-only` condition (plateau detector + corpus snapshot only, no LLM, no recipe-guided mutator, no Ghidra). The venue version will add it. Without it we cannot fully separate "controller machinery" from "default rule recipe", and the controller-driven attribution above remains a hypothesis rather than a measured effect. The full multi-arm comparison (`baseline` / `rule-only` / `no-mutator` / `no-static-analysis` / `controller-only` / `full-agent`) at $N \geq 10$ on a non-saturated target is explicit follow-up work (§8).

E4 fairness baselines: AFL++ with dict / cmplog (N=3). The preprint `baseline-afl` arm is vanilla AFL++ without `-x dictionary` or `cmplog` instrumentation (§8), so any plateau delta against it potentially overstates FuzzPilot's advantage. E4 is the corresponding fairness controls, which evaluate alternative uses for the same target-specific information or runtime cost. `baseline-afl-dict` runs vanilla AFL++ with `-xexperiments/targets/cjson/fuzzpilot_default.dict` (the same `FUZZ`/`MAGIC`/`TOKEN` tokens the controller emits in its default recipe), and `baseline-afl-cmplog` runs vanilla AFL++ over a `cmplog`-instrumented build (`cjson_fuzzer.cmplog`). 3 runs each, 14,400 s, same host. Two findings stand out:

- **Plateau: AFL++ with `-x` or `cmplog` does *not* close the plateau gap.** Medians are `baseline-afl-dict` = 2,256 s (p=0.79, A_{12}=0.60 vs baseline) and `baseline-afl-cmplog` = 2,367 s (p=1.00, A_{12}=0.53 vs baseline) — neither is meaningfully different from the 2,532 s vanilla baseline, and both are much longer than the FuzzPilot ablation arms (range 234–1,384 s, though note E2c's bimodal distribution). The descriptive plateau-reduction pattern we observe in §6.5 *does not reduce* to "vanilla AFL++ once it has a dictionary" or "vanilla AFL++ once it has runtime input-to-state-correspondence (I2S) information", suggesting the controller machinery (if our hypothesis is correct) operates via a different mechanism.
- **Edges: `cmplog` reaches one more edge than FuzzPilot on cJSON.** All three `baseline-afl-cmplog` runs hit 270 edges (vs 269 for `baseline-afl`, `full-agent`, and all E2 ablations; one `baseline-afl-dict` run also hits 270). The +1-edge gap is small but reproducible across all 3 `cmplog` runs and **must be disclosed**: `cmplog`'s runtime comparison-operand mining finds an edge that FuzzPilot does not. This is a **negative result for FuzzPilot on the absolute-coverage axis**. The two techniques operate on different axes: FuzzPilot's controller (hypothetically) shortens plateau duration via corpus reorganization (wall-time

advantage), while `cmplog` extends absolute coverage via I2S mining (coverage advantage). They are complementary, not substitutes. Composing the two (FuzzPilot's controller + `cmplog` instrumentation) is an obvious follow-up that the current FuzzPilot CLI does not yet support.

Interpretation: The plateau-reduction pattern we report in §6.2 and §6.5 is a descriptive wall-time advantage on a saturated target, not an absolute-coverage advantage. On cJSON, `cmplog` achieves better coverage (270 vs 269 edges) without shortening the plateau, while FuzzPilot (hypothetically via the controller) shortens the plateau without improving coverage. On a non-saturated target, the relative value of these two properties is an open question.

6.6 Headline summary (consolidated)

For a single-glance reading, Table 11 pulls together every headline RQ number, with the matching significance companion or null-result marker where applicable. Cells marked *desc.* are descriptive point estimates at the listed N; p is the two-sided Mann–Whitney U test (§6.2) unless noted; CI is the percentile bootstrap 95% over 10,000 resamples. Negative findings (zero promotions, TOST not established, etc.) are explicitly listed rather than buried.

RQ	Metric	Result (N)	Companion / caveat
RQ1	median plateau, baseline	2,532 s (N=5)	CI [539, 3,970]
RQ1	median plateau, full-agent	1,384 s (N=5)	CI [238, 3,226]
RQ1	Δ plateau (full vs baseline)	-45.3% (*desc.*)	MW U=8, p=0.42, A_{12}=0.68; **not significant at** N=5
RQ1	median time-to-269 edges, baseline	2,524 s (N=5)	range [1,081, 4,448]
RQ1	median time-to-269 edges, full-agent	6,702 s (N=4/5)	one run never reaches 269 within budget
RQ2	median exec/sec ratio (full / baseline)	$\approx 1.06\times$ (N=5)	leave-one-out (drop r05): 1.05×
RQ2	microbench fp-active / fp-empty, median	1.016× (N=5)	**mean ratio** 0.87×; both within noise
RQ2	microbench fp-active / fp-empty, t-test	two-sided p=0.45	no significant diff at N=5
RQ2	TOST equivalence at ±5%	p_{TOST}=0.68 (N=5)	**equivalence not established at** N=5
RQ2	microbench fp-empty / vanilla, median	1.53× (N=5)	sanity gate ($\leq 5\times$), passes wide
RQ3	schema-valid LLM responses	43/45 = 95.6% (N=5)	rule-only / no-mutator: 26−27/27
RQ3	LLM-recipe promotions (gate accepted)	0/20 (N=5)	cJSON saturated, R=0 on every micro-campaign
RQ3	DeepSeek API cost	\$0.002 / 4h campaign	FX rate \$1≈¥7.2; compute ∼\$1.60/campaign
RQ4	median plateau, rule-only (E2a)	1,165 s (N=3)	MW U=4, p=0.39, A_{12}=0.73
RQ4	median plateau, no-mutator (E2b)	930 s (N=3)	MW U=2, p=0.14, A_{12}=0.87
RQ4	median plateau, no-static-analysis (E2c)	234 s (N=3)	MW U=5, p=0.57, A_{12}=0.67; bimodal [187, 234, 4,454]
RQ4	ablation ordering	**no-static-analysis < no-mutator < rule-only < full-agent**	**all four shorter than baseline; controller-driver hypothesis**
RQ4	controller-only ablation	—	not in preprint matrix; venue version
Fair	E4a median plateau, baseline-afl-dict	2,256 s (N=3)	p=0.79, A_{12}=0.60 vs baseline; dict alone does not help
Fair	E4b median plateau, baseline-afl-cmplog	2,367 s (N=3)	p=1.00, A_{12}=0.53 vs baseline; cmplog does not shorten plateau
Fair	E4b edges, baseline-afl-cmplog	270/270/270	**cmplog finds +1 edge over FuzzPilot's** 269 (different axis, complementary)

Table 11: One-glance summary of all RQ-level numbers reported in the preprint, with the matching significance / null-result companion. *Desc.* = descriptive point estimate; no pairwise plateau or last_find difference reaches p<0.05 at this N. Reproducible from `scripts/paper01/review_stats.py`.

7 Related Work

In-loop LLM fuzzers. LLAMAFUZZ [20] uses an LLM fine-tuned on seed/mutation pairs to produce structurally valid mutations, and reports consistent gains over AFL++ on Magma. The model is queried per mutation; throughput is bound by model latency. FuzzCoder [14] casts byte-level mutation as a sequence-to-sequence task. Fuzz4All [23] treats the LLM as the primary

input generator and mutation engine across multiple input languages and targets. Semantic-Aware Fuzzing [2] explicitly notes the throughput bottleneck of in-loop designs and offers it as motivation for future work. FuzzPilot diverges from this family by treating the LLM as a strategy proposer rather than a per-mutation step.

Off-hot-path placement: G^{2}FUZZ. G^{2}FUZZ [28] is the closest prior work and the one we explicitly position against. Both designs: invoke the LLM only when the fuzzer fails to make progress; have the LLM emit a reusable intermediate rather than per-input mutations; and let AFL++ run at native speed afterwards. The differences are intentional and orthogonal:

- **Target domain.** G^{2}FUZZ targets non-textual binary formats (JPEG, TIFF, MP4, ...); FuzzPilot is *designed for* structured-text parsers and *this preprint evaluates only cJSON*. Generalization claims (XML, SQL, libpng, Magma suite) are deferred to the venue paper.
- **Intermediate representation.** G^{2}FUZZ emits Python generator scripts (code); FuzzPilot emits schema-validated recipes (data). §4.5 compares the two.
- **Validation.** G^{2}FUZZ's generated inputs enter the AFL++ queue and are filtered by coverage feedback as usual; FuzzPilot's recipes are pre-promotion-gated by an explicit N-second micro-campaign with a reward function.
- **Static context.** G^{2}FUZZ uses an LLM-extracted "feature list" from format documentation; FuzzPilot uses Ghidra headless on the target binary (function summaries, dictionary candidates, protected ranges), supporting closed-source targets.

We do not attempt a head-to-head numerical comparison against G^{2}FUZZ in this preprint. Their evaluation is on non-textual binary formats outside our scope; ours is on structured-text parsers outside theirs. §8 states this scope split explicitly.

Off-hot-path on language processors: HLPFuzz. HLPFuzz [26] (USENIX Sec '25) also keeps the LLM out of the per-mutation hot path, but uses the model as a *constraint solver* invoked when the fuzzer needs to traverse a multi-stage language-processor pipeline (lexer → parser → semantic check → codegen) where the constraints required to reach deep program states are authored in the target's own source. The authors introduce *hybrid centrality prioritization* and *iterative context construction* to keep the LLM call rate low, and report +190% code coverage over the second-best fuzzer on 9 popular language processors (compilers and interpreters) with 52 bugs found (37 confirmed, 14 fixed) and 5× more bugs than the runner-up. HLPFuzz is the most direct competitor to FuzzPilot's *intended* domain — structured-text and language processors — but the artifacts differ in kind: HLPFuzz emits concrete input fragments that satisfy a specific constraint, while FuzzPilot emits a schema-validated *recipe* that re-tunes the host mutator's operator weights, dictionary, and protected ranges at the byte level. We do not run a head-to-head comparison in this preprint because FuzzPilot is evaluated on cJSON only, which is a single-stage tokenizer plus recursive parser without the deep constraint pipeline that HLPFuzz targets; a true comparison requires the language-processor benchmark used in HLPFuzz [26] and is the single most informative deferred experiment for the venue paper.

Hybrid static-and-LLM fuzzers. The hybrid fuzzer of Lin et al. [12] integrates static analysis (CFG and data-flow extracted via LLVM IR or angr) with LLM-guided input mutation through a helper fuzzer running at ≈ 250 queries/hour. It also reports a syntax-schema validator and a

semantic novelty score. FuzzPilot differs in two ways: (a) the LLM is genuinely off the hot path, not on a parallel hot path, and (b) the validation step is a budgeted AFL++ run with a reward function, not an embedding-distance score against past inputs.

LLM-synthesized mutators for compilers. MetaMut [15] and Mut4All [27] use LLMs to synthesize *static* custom mutators (in C/C++) for compiler fuzzing, compiled once and used many times. FuzzPilot's recipes are dynamic per-plateau artifacts rather than compiled mutators, and target consumer parsers rather than compilers.

Multi-agent fuzz frameworks. MALF [1] introduces a multi-agent LLM framework for industrial control protocol fuzzing with RAG and QLoRA fine-tuning. The role split is similar in spirit to FuzzPilot's coordinator / plateau-diagnosis / scheduler / cmp-hint / dictionary / format / mutator / corpus / crash-triage agents; the domain (industrial protocols vs structured text parsers) and the intermediate representation (input sequences vs recipes) differ.

LLM-as-generator (whole inputs). TitanFuzz [6] demonstrates that an LLM can synthesize *entire program-level inputs* (Python snippets exercising deep-learning libraries) zero-shot, and that infill-style prompting matches or exceeds template-driven generators on coverage. FuzzGPT [7] extends the idea by prompting the LLM with historical bug-triggering inputs to push it toward edge cases. ChatFuzz [9] couples ChatGPT-style mutations with greybox feedback for structured-text formats (JSON / XML / HTML) and is the closest single-target comparator to FuzzPilot's structured-text scope. All three generate *inputs* directly; FuzzPilot's intermediate is a *recipe* that a downstream AFL++ mutator consumes, so the LLM inference cost is amortized across the entire plateau interval rather than per input.

LLM with white-box / static context. WhiteFox [24] drives compiler fuzzing with LLM-synthesized programs that target specific compiler-optimization rules extracted from source by another LLM pass; the white-box context source is the compiler's own implementation. KernelGPT [25] mines kernel headers and source to build syscall specifications and then has an LLM synthesize Syzlang-like input programs. FuzzPilot's Ghidra channel (§5.2) is a parallel design point that opts for binary-level static analysis (so the same pipeline applies to closed-source targets) rather than source-level extraction; the trade-off is coarser per-function summaries in exchange for applicability beyond the open-source assumption.

Industrial LLM-driven fuzzing infrastructure. OSS-Fuzz-Gen [13] is Google's production framework for using LLMs to *generate fuzz targets and harnesses* for the OSS-Fuzz fleet, addressing a complementary problem (harness/target authoring at scale) to FuzzPilot's (in-campaign recipe proposal). The two could be composed: an OSS-Fuzz-Gen-authored harness fed into a FuzzPilot-orchestrated campaign with a Ghidra-informed recipe layer. We do not evaluate this composition here.

Grammar / AST-aware greybox fuzzers (LLM-free). A second family of structured-text fuzzers does not use LLMs at all and obtains its structure awareness from an explicit grammar or AST. AFLSmart [16] extends AFL with an input-model-aware mutation phase that preserves chunk boundaries; Nautilus [4] drives greybox mutation from a context-free grammar with subtree splicing; Superion [21] pairs AFL with an ANTLR-derived AST-mutation operator. Conceptually FuzzPilot's recipe vocabulary (§4.2) is byte-level rather than AST-level: a recipe can express "insert dictionary

token at offset x", not "replace the second `value` child of an `object` node". We acknowledge this as a design limitation (§8) — on grammar-amenable targets, AFLSmart-style chunk-aware mutation or Nautilus-style subtree splicing remains a strong baseline that we do not attempt to outperform in this preprint. Whether an LLM-guided recipe layer composes *on top of* a grammar-aware mutator (e.g., FuzzPilot recipes that target Nautilus subtrees rather than byte offsets) is an explicit follow-up direction.

AFL++ semantic-ish features (cmplog, RedQueen, laf-intel). A third class of techniques exploits the AFL++ instrumentation itself rather than an external grammar or LLM. The `cmplog` module records concrete comparison operands during fuzzing and feeds them back as dictionary-overwrite candidates; RedQueen [5] pioneered the underlying input-to-state-correspondence (I2S) idea; the `laf-intel` LLVM pass [11] splits multi-byte comparisons into single-byte ones to make magic numbers reachable by coverage feedback alone. These are precisely the AFL++ features a fair `baseline-afl` arm should engage. Our preprint `baseline-afl` arm *does not* pass `-x dictionary` or build the target with `cmplog`, and so the plateau-delta measured against it potentially overstates FuzzPilot's advantage. We treat this as the single most important missing baseline (§8); the E4 fairness arms reported in this paper (`baseline-afl-dict` with the FuzzPilot default dictionary, and `baseline-afl-cmplog` with a `cmplog`-instrumented `cjson_fuzzer.cmplog`) close this directly — and find that neither closes the plateau gap, but `cmplog` reaches +1 edge that FuzzPilot does not (see §6.5).

LLM-upfront generators (one-shot, no steady-state LLM calls). Sphinx [18] uses an LLM *once, before fuzzing starts* to synthesize a structured input generator for an SMT solver; the generator then runs without any further LLM involvement. The design rhymes with FuzzPilot in placing the LLM outside the per-mutation hot path, but the LLM is consulted only once (not on every plateau) and the output is a complete generator rather than a recipe that re-tunes a host fuzzer's existing mutation operators. Sphinx and FuzzPilot together suggest a spectrum — *LLM once* (Sphinx), *LLM at plateaus* (FuzzPilot, G^{2}FUZZ), *LLM per input* (LLAMAFUZZ, ChatFuzz) — that we expect to be a productive axis for future comparisons.

T3: side-by-side comparison. Table 12 summarizes the comparison axis-by-axis.

System	LLM placement	LLM output	Validation	Static ctx.	Audit	Target domain
AFL++ [8]	*none*	—	coverage	none	—	general
AFL++ `cmplog` / RedQueen [5]	none (I2S)	dict override	coverage	runtime cmp ops	—	general
laf-intel [11]	none (LLVM pass)	—	coverage	cmp splitting	—	general
AFLSmart [16]	none (input model)	chunk mutation	coverage	input-format model	no	structured (PNG / WAV / PDF)
Nautilus [4]	none (grammar)	subtree splicing	coverage	context-free grammar	no	grammar-amenable (JS / SQL / PHP)
Superion [21]	none (grammar)	AST mutation	coverage	ANTLR grammar	no	structured text (XML / JS)
Sphinx [18]	**LLM upfront, one-shot**	input generator (code)	coverage	none	no	SMT solvers
LLAMAFUZZ [20]	in-loop	input bytes	coverage	none	no	general (Magma suite)
FuzzCoder [14]	in-loop	byte mutation	coverage	none	no	general
Fuzz4All [23]	in-loop	input bytes	coverage	none	no	multi-language
SemAware [2]	parallel helper	input bytes	semantic novelty	none	no	general
Hybrid [12]	parallel helper, ≈ 250 rph	input bytes	novelty + syntax	LLVM-IR / angr	no	libpng / tcpdump / sqlite
TitanFuzz [6]	generator (zero-shot)	whole program input	coverage	none	no	DL libraries (PyTorch / TF)
FuzzGPT [7]	generator (history-prompted)	whole program input	coverage	none	no	DL libraries
ChatFuzz [9]	in-loop (ChatGPT mutator)	input bytes	coverage	none	no	structured text (JSON / XML / HTML)
WhiteFox [24]	two-stage generator	whole program input	coverage	source-level (compiler rules)	no	compilers (PyTorch / LLVM)
KernelGPT [25]	generator + spec-mining	syscall sequences	coverage	source-level (kernel)	no	kernels (Linux)
OSS-Fuzz-Gen [13]	offline (target authoring)	fuzz harness code	build + coverage	source-level (OSS repos)	no	OSS-Fuzz fleet
MetaMut [15]	offline	C/C++ mutator	manual	none	no	compilers
Mut4All [27]	offline	mutator code	manual	bug reports	no	compilers
MALF [1]	multi-agent in-loop	input seqs	coverage	protocol KB	no	industrial protocols
G^{2}FUZZ [28]	**off-hot-path**	Python generator (code)	downstream coverage	LLM-extracted features	no	non-textual binary (JPEG / TIFF / MP4)
HLPFuzz [26]	**off-hot-path** (constraint solve)	input fragments + constraint witnesses	downstream coverage	source + grammar (hybrid centrality)	no	language processors (compilers / interpreters)
FuzzPilot (ours)	**off-hot-path**	**schema-validated recipe (data)**	***N*-sec micro-campaign**	**Ghidra headless**	**yes**	**cJSON (preliminary; designed for structured-text parsers)**

Table 12: Comparison across LLM-augmented and LLM-adjacent fuzzers. FuzzPilot is positioned as a sibling of G^{2}FUZZ in the off-hot-path quadrant rather than a head-on competitor; the row-by-row differences in intermediate (data vs. code), validation (micro-campaign vs. downstream coverage), context source (Ghidra vs. LLM-extracted features), and target domain (structured text vs. non-textual binary) establish the contribution.

8 Limitations, Threats to Validity, and Future Work

Summary of limitations. This version has narrow empirical support. The main limitations are:

1. **Single saturated target**: cJSON is the only evaluated target, and it saturates at 269 edges well before the budget ends, leaving no headroom for the proposal layer or micro-campaign gate to show coverage value. The architecture's utility on non-saturated targets is untested.

2. **Model proposal layer not yet shown to help**: Zero of 20 model-proposed recipes were promoted on cJSON due to target saturation (§6.4). The micro-campaign gate's intended property—distinguishing good recipes from bad—remains untested.

3. **Attribution incomplete**: Ablation experiments suggest the controller's plateau-detection machinery is the primary driver, not the recipe-guided mutator or LLM layer. However, the `controller-only` ablation required to test this hypothesis is not in the preprint matrix (§6.5).

4. **Statistical power insufficient**: At N=5 (main) and N=3 (ablations), no pairwise difference reaches p<0.05. All findings are descriptive point estimates, not significance claims.

5. **Negative result on absolute coverage**: AFL++ with `cmplog` instrumentation reaches 270 edges (vs FuzzPilot's 269) on cJSON, indicating FuzzPilot does not improve absolute coverage on this target (§6.5).

Given these constraints, this preprint should be read as an implementation report with a cJSON proof-of-concept, not as evidence that all components help on all targets. The detailed limitations follow below.

Scope: cJSON only in this preprint. This preprint evaluates FuzzPilot on cJSON only (5 `baseline-afl` + 5 `full-agent` runs at 14,400 s, plus the E3 microbench and 3 preliminary runs each for E2a/E2b/E2c ablations, plus 3 runs each for E4a/E4b fairness baselines). The architecture is designed for structured-text parsers, but no empirical claim is made about libpng, libxml2, sqlite3, openssl, or any non-textual binary format here; on those domains G^{2}FUZZ's code-as-generator is plausibly stronger. A head-to-head comparison on G^{2}FUZZ's benchmark (JPEG, TIFF, MP4, etc.) and the larger structured-text targets are deferred to the venue version of the paper.

cJSON is a saturated target for the proposal layer. Within the 14,400 s budget the baseline AFL++ harness reaches the 269-edge ceiling at a per-run median of 2,524 s (slowest baseline run 4,448 s), leaving the parent corpus at plateau time with no edge headroom for a 20 s micro-campaign to find. As a result, the proposal layer's contribution (the *model-promoted recipe* path, distinct from the *mutator framework*) cannot be empirically demonstrated on this target: every micro-campaign reward is R=0 and every promotion is skipped (§6.4). A non-saturated target where the gate has edge headroom is required to evaluate that layer in isolation; this is the most important single piece of follow-up work.

Ghidra benefit not fully demonstrated. Our targets are open-source; LLVM-IR or angr-based static analysis would, in principle, supply the same blackboard fields used here. The motivation for Ghidra is to make the architecture applicable to closed-source binary targets, and the E2c ablation (§6.5) only quantifies the contribution of static context, not the specific advantage of Ghidra over alternatives. A binary-target evaluation that isolates Ghidra's contribution is explicit future work.

Repetition count and statistical power. We run 3–5 repeats per cell, which is consistent with prior LLM-fuzzing preprints [12, 20, 28] but insufficient for formal significance claims. We report medians, interquartile ranges, per-run raw numbers, Mann–Whitney U p-values, Vargha–Delaney A_{12} effect sizes, and bootstrap CIs to make the uncertainty explicit. The venue paper will use $N \geq 20$ main runs and $N \geq 10$ ablation runs with rank-sum tests.

Crash deduplication not in scope. We report coverage and plateau-recovery, not unique-bug counts. Crash deduplication (e.g. the casr-cluster pipeline) is deferred.

Deferred ablations and sensitivity. The `controller-only` ablation, the `ai-direct` ablation (promote every schema-valid proposal without micro-campaign), the `random-recipe` ablation (skip the LLM entirely and emit randomly-sampled recipes), and a micro-campaign-parameter sensitivity sweep (over N, K, and the reward weights $\alpha, \beta, \gamma, \delta_h, \delta_m$) are not part of the preprint matrix. E2a/E2b/E2c and E4 are already reported in §6.5; the deferred arms are needed to close attribution and hyperparameter-sensitivity questions in the venue version.

Recipe vocabulary is byte-level, not AST-level. The seven-operator recipe vocabulary (`BitFlip`, `OverwriteRange`, `InsertToken`, `Arith`, `Splice`, `DeleteBlock`, `DictionaryOverwrite`; §4.2) operates on *byte offsets within a flat input buffer*, not on syntactic nodes of the structured-text grammar. A recipe can say "insert the token `"null"` at offset 128"; it cannot say "replace the second child of an `object` node with `null`". For grammar-amenable targets, AST-level operators (Nautilus [4], Superion [21]) or chunk-level mutations (AFLSmart [16]) are stronger generators of parser-valid mutants. We chose a byte-level vocabulary deliberately — to keep the mutator hot-path simple and to keep the recipe schema small enough for the LLM to emit reliably — but the trade-off is real, and FuzzPilot is unlikely to *outperform* a grammar-aware fuzzer on grammar-amenable targets without additional integration. Composing FuzzPilot's recipe layer on top of a grammar-aware mutator (recipes that target AST subtrees rather than byte offsets) is an explicit future-work direction.

No syntax-aware repair or invariant-preserving operators. Several recipe operators (`OverwriteRange`, `DictionaryOverwrite`, `Splice`) can produce ill-formed JSON if applied naively: an overwrite that straddles a string-literal boundary breaks the quote-pair invariant; a splice that lands inside a numeric literal can produce an unparseable token; a dictionary overwrite inside a key context can violate the keys-are-unique invariant. The current recipe schema has *no length-preserving, quote-aware, or escape-aware* repair operator. AFL's downstream coverage feedback eventually discards inputs that fail to advance coverage, so the practical impact on the campaign is bounded; but a sharper grammar-aware mutant generation strategy is the obvious next step. We have considered adding a `LengthRepair` operator (rebalance quote/bracket pairs after a mutation) and a `StringInsert` operator (insert at safe byte boundaries within a string literal), but neither is implemented in the released version — avoiding having to specify them is part of why we kept the schema small and audit-friendly. We acknowledge this is a real reviewer-grade limitation rather than a design victory.

AFL++ fairness baseline (`-x dictionary` and `cmplog`). The `baseline-afl` arm reported in §6.2 is *vanilla* AFL++: it does *not* receive an `-x dictionary` file, and the target binary is *not* compiled with the `cmplog` pass. FuzzPilot's recipe-guided mutator, by contrast, dispatches against the controller's default `DictionaryAgent` recipe whose token set is {`"FUZZ"`, `"MAGIC"`, `"TOKEN"`} (§6.4). This asymmetry overstates FuzzPilot's advantage: any plateau-recovery delta could in principle be closed by an AFL++ run that consumes the *same* three tokens via `-x`, or by an AFL++ build that uses `cmplog`/RedQueen-style I2S mining [5, 11] to infer target-specific tokens at runtime. This asymmetry is why the E4 fairness baselines (`baseline-afl-dict` with the FuzzPilot default dictionary; `baseline-afl-cmplog` with a `cmplog`-instrumented `cjson_fuzzer.cmplog`) are reported in §6.5; the headline finding is that neither closes the plateau gap, but `cmplog` reaches one more edge (270) than any FuzzPilot arm (269). §9 pins the dict file and the cmplog binary so reviewers can re-run the comparison themselves.

Internal code audit: implementation findings (A1, A2, A4, A5). While preparing the venue-extension matrix we ran a focused code-level audit of the FuzzPilot controller, custom mutator, and agent runtime. The audit surfaced four implementation findings that the cJSON-only evaluation cannot expose but that materially shape how the preprint numbers should be read and how the deferred multi-target experiments are interpreted. We disclose them in full so that reviewers can stress-test our claims against the source. (Finding "A3" — the missing `cmplog`/dictionary fairness baseline — is covered in the previous paragraph.)

A1: Plateau detector is single-shot and gated by `last_path` staleness. The detector in `src/plateau/detector.cp` is guarded by an internal `emitted_` flag that turns true on the first qualifying event and is only cleared by an explicit `reset()` call from the controller. It additionally requires that AFL++'s reported `last_path` timestamp has aged out by the full window (the `stale_last_path` gate at `src/plateau/detector.cpp:92--93`). On cJSON this behavior is benign because the 269-edge ceiling is reached well before the window expires and `last_path` naturally stales; the preprint's cJSON plateau measurements are unaffected. On a high-yield target such as `libxml2` ($\sim$6.6 s per new edge in pilot smoke runs), `last_path` refreshes continuously and the natural plateau never fires. To compensate, the controller (`src/controller/run.cpp`) carries an `agent_heartbeat_sec` mechanism that synthesizes a plateau event after a configurable interval; the agent layer's behavior on non-saturated targets is therefore a function of the heartbeat rate rather than the natural detector. Reviewers should not transfer the cJSON-trace single-plateau pattern to other targets without checking the heartbeat configuration.

A2: Agent calls are sequential per plateau. The eight default agents (`CoordinatorAgent`, `PlateauDiagnosisAgent`, `SchedulerAgent`, `CmpAgent`, `MutatorAgent`, `DictionaryAgent`, `FormatAgent`, `CorpusAgent`) are invoked in a sequential `for`-loop in `src/agents/agent_runtime.cpp::run_agent_tasks`. The `agent_runtime.max_parallel_model_calls` configuration field is declared in `include/fuzzpilot/config.hpp` but is not consumed by the agent runtime in this release. With the default 30 s per-agent timeout, the worst-case wall-clock for a full plateau response is bounded by $8 \times 30\,\text{s} = 240\,\text{s}$ of model-bound time before any mutator weights are updated. The intra-plateau deadline (`deadline_unix_sec` in `run_agent_tasks`) prevents this from cascading past the plateau budget but does not eliminate per-plateau blocking. On saturated targets like cJSON the absolute number of plateau events is small and the sequential cost is hidden; on a multi-plateau target it becomes the dominant scheduling constraint, and an honest interpretation of throughput parity must factor in the sequential block. The venue evaluation will report the realized parallelism factor and a parallel agent runtime if the measurement justifies the engineering.

A4: Recipe-miss fallback uses a 3-operator simplified havoc. When the recipe cache has no recipe for the current seed (`recipe_hit=false`), the FuzzPilot custom mutator dispatches against a hard-coded `global_` default of {`BitFlip`:0.45, `OverwriteRange`:0.35, `Arith`:0.20} (`mutators/fuzzpilot/recipe_cache` AFL++'s native havoc stage uses on the order of 15 stacked operators including `splice`, `interest`-value substitution, dictionary overwrites, and havoc-stacking; the FuzzPilot fallback path is therefore *strictly weaker* than native havoc on any seed that the LLM has not yet covered with a recipe. This is the leading code-level hypothesis for why the `full-agent` arm can underperform `baseline-afl` on non-saturated targets: until recipe coverage is wide enough, the fallback path *is* the mutator's main hot path. The deferred `controller-only` ablation (§6.5) — in which the FuzzPilot plateau detector and agent layer run but the custom-mutator stage is disabled so AFL++ uses native havoc — is designed to isolate exactly this contribution and is the highest-priority ablation in the venue matrix.

A5: `baseline-afl` and `full-agent` differ by two environment variables. For transparency: at process-launch time the only difference between the two modes is whether `AFL_CUSTOM_MUTATOR_LIBRARY` and `FUZZPILOT_RECIPE_STORE` are exported into AFL++'s environment (`src/runner/afl_runner.cpp`,

under the `config.mutation_strategy.enabled` branch). The full-agent pipeline (plateau detector, agent runtime, micro-campaign, recipe store) is then *controller-side* machinery that mutates the recipe store while AFL++ runs. This means that any `full-agent` versus `baseline-afl` delta is mechanistically attributable to (i) the recipe-cache custom mutator (A4 above), (ii) any controller-driven corpus/seed-energy reorganization, or (iii) the env-var presence itself triggering an AFL++ codepath change — and *not* to any per-mutation LLM call, which by design never happens. We surface this so that the "LLM-in-the-loop" framing common in prior LLM-augmented fuzzing work is not silently assumed about FuzzPilot.

These four findings do not invalidate the cJSON numbers reported in this preprint — our null-result reporting is consistent with A1/A2/A4 and the cJSON evaluation runs to its 269-edge ceiling within budget — but they reshape what the cJSON null tells us about the *architecture* and what the deferred multi-target experiments need to disambiguate. We surface them in full rather than wait for the venue extension to make the implementation risk visible.

Reward / hyperparameter sensitivity is fixed, not swept. The micro-campaign budget N=20 s, the candidate count K_{cand}=4, and the reward weights $(\alpha, \beta, \gamma, \delta_h, \delta_m)$ in §5.3 are pinned to fixed values for the preprint matrix — chosen during development on the cJSON corpus, then frozen. We did not run a sensitivity sweep. The risk surface that this exposes is twofold. *(i)* N=20 s may be too short for the gate to discriminate marginal candidates on a non-saturated target (the cJSON null result is agnostic on this since reward is identically zero). *(ii)* The reward weights bias the gate toward edge-discovery; on a crash-finding-dominant target, a different $(\alpha, \delta_h, \delta_m)$ trade could matter. A small sensitivity sweep (e.g., $N \in \{10, 20, 40, 80\}$ s and $K_{\text{cand}} \in \{2, 4, 8\}$ at N=3 each) is wired into the CLI and is part of the venue-paper matrix.

Multi-target generalization. We do not run on Magma's full benchmark suite in this preprint, and we do not claim generalization across target families. The ongoing evaluation adds three deeper structured-text / binary-schema targets that exercise components of FuzzPilot that cJSON's 269-edge ceiling does *not*: `libxml2` (XML parser, ∼30k edges, AFLSmart/Nautilus/Superion comparator), `sqlite3` (SQL parser/VM, ∼150k edges, OSS-Fuzz flagship), and `openssl_x509` (ASN.1 + X.509 parser, ∼80k edges, binary + deep-schema). The larger evaluation matrix (`experiments/manifests/paper01_venue.yaml`) follows Klees et al. [10] at $N \geq 20$ main runs and $N \geq 10$ ablation runs per target, 24 h budget per run (Magma/OSS-Fuzz convention). A Stage-1 pilot (N=5 main / N=3 ablations, 24 h budget) is scheduled on the experiment host with the harnesses (`libxml2_fuzzer`, `sqlite3_fuzzer`, `x509_fuzzer`) and their per-target dictionaries (`extracted.dict`, ∼1,000–6,000 entries per target; see Table 1) checked into the repository; the Stage-1 outcome will decide whether the full Klees-style matrix runs on all three targets or focuses on the subset where the proposal layer shows a non-null contribution. *Headroom check:* a 60 s smoke-test run of vanilla `afl-fuzz` on the three harnesses (no FuzzPilot pipeline) reaches 2,776 edges on libxml2, 6,922 edges on sqlite3, and 3,635 edges on openssl X.509 — 10–25× more than cJSON's 269-edge ceiling at the *end* of a 14,400 s budget. The 24-hour runs should leave room for the proposal layer to produce a non-null result if it can. The choice to publish this first cJSON-only version is deliberate: it lets readers inspect the audit trail, gate, mutator, and micro-campaign machinery on a small target before the larger runs finish. *No claim in this version depends on these larger targets*; their data will be reported separately after the runs finish.

Platform. The canonical evaluation platform is the native Linux/x86_64 fuzz cloud server described in Table 2. Containerized execution is not the reproducibility path for this preprint. All numbers in

this paper are from the native fuzz-server build and run artifacts.

LLM nondeterminism. Even with temperature pinned at 0.0, model providers do not guarantee bit-exact reproducibility across versions. We mitigate by (a) recording the audit trail so any decision is replayable from the recipe rather than the model, (b) reporting schema-validity and fallback rates per run, and (c) recording the model identifier, prompt/response hashes, run-time commit, and host toolchain metadata in the native fuzz-server artifacts.

Threats to validity. We organize the threats following Wohlin et al.'s [22] standard internal / external / construct / conclusion taxonomy, with an additional explicit discussion of LLM-specific risks.

Internal validity. The plateau-detector hyperparameters (W=10 s, θ_e=50, θ_p=1) and the micro-campaign reward weights ($\alpha, \beta, \gamma, \delta_h, \delta_m$) were tuned on a *development cJSON seed corpus* (the 26 hand-curated JSON files described in §6.1) that became the *evaluation* seed corpus once the design was frozen. We did not implement a hold-out split between tuning and evaluation corpora and therefore cannot exclude data leakage; the venue version will tune on a disjoint development corpus or via cross-validation across seed-corpus splits, and will report the delta from tuning-set to evaluation-set numbers. A second internal threat is the absence of a `controller-only` ablation (§6.5) that would separate the plateau detector / corpus-snapshot reorganization machinery from the default rule recipe; we note this as a known design gap to be addressed in the venue ablation matrix. Plateau-detector SIGSTOP-based microbench isolation (§5.3) is also a methodological hazard: running the microbench concurrently with the main AFL workers *without* SIGSTOP inverts our central RQ2 throughput-parity ratio in pilot tests, so any reproducer that omits the isolation will obtain different numbers.

External validity. A single target family (cJSON, parser of a structured-text format) is not representative; we make *no* cross-format generalization claim and explicitly defer it. The LLM proposal layer in particular is exercised but not stress-tested by the cJSON target: the gate returns 20/20 null candidates (§6.4), so we observe only the *no-false-positive* half of the gate's intended behavior. A single LLM (`deepseek-chat`) at temperature 0 is also a single-point evaluation in model space; whether `claude-haiku-4-5`, `gpt-4o-mini`, `llama-4-maverick`, or any other backend produces a materially different schema-validity rate, recipe distribution, or null-result behavior is open. Finally, all numbers were collected on a single 4-vCPU experiment host (single tenancy at collection time except where noted in §6.3); cross-machine reproducibility is not separately measured. The released artifacts prioritize replaying and auditing the native fuzz-server runs rather than certifying bit-identical reproduction on new hardware.

Construct validity. The choice of metric is itself a construct decision. We use plateau duration (`run_time` − `last_find`) as a primary endpoint metric, which is sensitive to a single late-budget discovery (e.g. full-agent r04 finds the 269th edge at 14,205 s and is credited with a 238 s plateau; see Table 5). Klees et al. [10] recommend coverage-AUC or time-to-N-edges as endpoint-free alternatives; we therefore additionally report time-to-N-edges for $N \in \{264, 268, 269\}$ in Table 4, which gives the opposite headline (`baseline-afl` reaches the 269-edge ceiling *faster* than `full-agent` at the median). The two metrics are mutually consistent — a plateau-shortening intervention can also delay the ceiling-touch by spreading out the discovery curve — but the construct choice matters and we report both. The 269-edge "ceiling" is itself a construct: it is the number of AFL++ bitmap slots hit by the cJSON harness, not a measure of all reachable basic-block edges in the cJSON library; a richer harness (or a different fuzzing instrumentation) could expose more edges.

Conclusion validity. With N=5 per main arm and N=3 per ablation, we are well below

the $N \geq 20$ guideline of Klees et al. [10]. We therefore report all group differences as descriptive point estimates accompanied by Mann–Whitney U, exact p-values, Vargha–Delaney A_{12} [19], and percentile-bootstrap 95% confidence intervals (see §6.2). No descriptive delta in this preprint reaches $p<0.05$ on the two-sided Mann–Whitney test; we present each result as a hypothesis rather than as a tested claim and the venue version will repeat the experiments at $N \geq 10$ with formal significance testing.

LLM-specific reproducibility. The model identifier `deepseek-chat` is a moving alias served by DeepSeek; the underlying weights may change without notice between runs. At $T=0$ the API surface is intended to be deterministic, but our data shows 2/45 schema-invalid responses, indicating that the API surface is not bit-exact deterministic in practice. We mitigate by (a) logging the `model` field returned in each API response into `agent_decisions.jsonl`, (b) recording the full prompt / response pair so any decision is replayable from the recipe rather than the model, and (c) recording the native fuzz-server toolchain and model-client configuration in the run artifacts. A future-proof reproducibility strategy would also archive the exact prompt-response transcripts in a content-addressable store to allow regeneration without re-querying the API.

9 Conclusion

FuzzPilot is a plateau-triggered controller for AFL++ structured-text campaigns. It represents intervention strategies as data recipes, tests candidate recipes in isolated micro-campaigns, and keeps the main mutation loop native. The cJSON evaluation in this first report is useful mainly because it exposes the behavior of that control path under a saturated target. Both `baseline-afl` and `full-agent` reach the same 269-edge ceiling within their 14,400 s budgets. The median throughput ratio (`full-agent`/baseline) is about 1.06×, and the median plateau is descriptively shorter under `full-agent` (1,384 s versus 2,532 s), but the difference is not statistically significant at $N=5$ (Mann–Whitney $U=8$, $p=0.42$, $A_{12}=0.68$).

The most important negative result is that no model-proposed recipe was promoted. Across five `full-agent` runs, the gate evaluated 20 candidates and rejected all of them because each validation reward was zero. That is the right conservative behavior on a saturated corpus, but it does not show that the proposal layer can improve coverage. The ablations point instead to the controller's snapshot and restart machinery as the likely source of the observed plateau reduction, and that interpretation remains only a hypothesis until a `controller-only` arm is run.

This version therefore makes a limited claim: the control path can be built, audited, and run without putting external reasoning in AFL++'s hot path. The larger claims require the ongoing non-saturated-target matrix, higher repetition counts, the missing `controller-only` ablation, and comparison against stronger format-aware baselines.

Reproducibility appendix

The numbers in this preprint are reproducible from the following native fuzz-server artifacts. All paths are relative to the repository root.

- **Experiment code commit:** `85223d844711c704130608f58200a0b5a18862fe` (branch `main`, tag `paper01-arxiv-v1`, repo https://github.com/Qiao-Zhiyi/fuzz_agent). This is the run-time code commit recorded in each completed campaign's `run_metadata.json`. The manuscript and aggregation text may include later paper-only edits; the raw fuzzing data are pinned by the per-run metadata and checked-in artifacts.

- **Fuzzer:** AFL++ build identifier `++4.21c` (the leading `++` is the upstream banner prefix; the underlying release tag is `v4.21c`). Built from source on the experiment host and confirmed in every `fuzzer_stats` file under `results/paper01_ai_recipe_mutation/runs/`.
- **Target binary:** cJSON parser harness at `experiments/targets/cjson/`, AFL persistent mode (`persistent shmem_testcase deferred`). The target binary SHA-256 and the seed-corpus tarball SHA-256 are written per-run into `run_metadata.json` (fields `target_sha256` and `seed_corpus_sha256`); reviewers can verify reproduction by comparing those hashes across re-builds.
- **Seed corpus:** 26 hand-curated JSON files at `experiments/targets/cjson/seeds/`; the directory is small enough to inspect directly and is included in the repository.
- **LLM:** `deepseek-chat` via the `api.deepseek.com` OpenAI-compatible endpoint (configuration in `experiments/targets/cjson/config.yaml`), temperature 0.0. *Note that `deepseek-chat` is a moving alias served by DeepSeek; the specific underlying model version is logged into the `model` field of each `agent_decisions.jsonl` record at call time, so a reviewer can recover the model-version string from the audit log.* A NIM-hosted `meta/llama-4-maverick-17b-128e-instruct` fallback path is wired through the same watchdog and was exercised in regression testing only — all numbers in this paper are from `deepseek-chat`.
- **Build mode:** native x86_64 Linux on the fuzz cloud server (no container in the hot path). Containerized execution is deprecated for this paper because the reported campaigns depend on the fuzz-server host toolchain, Ghidra installation, AFL++ build, CPU scheduling, and run directories captured in the native artifacts.
- **Per-run artifacts:** 25 completed 14,400 s runs (5 `baseline-afl`, 5 `full-agent`, 3 `rule-only`, 3 `no-mutator`, 3 `no-static-analysis`, 3 `baseline-afl-dict`, 3 `baseline-afl-cmplog`) are checked into the repository under `results/paper01_ai_recipe_mutation/runs/`; this is the complete preprint matrix on cJSON. Each run directory contains the unmodified AFL++ `fuzzer_stats`, `coverage.csv`, `events.jsonl`, `agent_decisions.jsonl` (full-agent / no-mutator modes), `main_launch.sh` (the exact command-line that started the AFL run), `run_metadata.json`, and `git.patch` (the working-tree diff at run start) for byte-level reproducibility of the experiment.
- **Per-mode launch commands.** Each ablation mode is selected by a single `--ablation` switch to the controller on the native fuzz-server environment; the canonical invocations are:

```
# E1a baseline-afl
python3 -m fuzzpilot.runner --target cjson \
--ablation baseline-afl --budget 14400 \
--run-id p1_e1_cjson_baseline-afl_r01
# E1b full-agent
python3 -m fuzzpilot.runner --target cjson \
--ablation full-agent --budget 14400 \
--run-id p1_e1_cjson_full-agent_r01
# E2a rule-only
python3 -m fuzzpilot.runner --target cjson \
--ablation rule-only --budget 14400 \
--run-id p1_e2_cjson_rule-only_r01
# E2b no-mutator
python3 -m fuzzpilot.runner --target cjson \
--ablation no-mutator --budget 14400 \
--run-id p1_e2_cjson_no-mutator_r01
# E2c no-static-analysis
python3 -m fuzzpilot.runner --target cjson \
--ablation no-static-analysis --budget 14400 \
--run-id p1_e2_cjson_no-static-analysis_r01
```

The host-wide orchestrator `scripts/paper01/run_all_host.sh --parallel 4` iterates the cJSON campaign matrix in 4-way parallel with the same per-mode invocations under the hood. The E4 dictionary and cmplog fairness runs are included in the checked-in artifact matrix and use the corresponding AFL++ launch commands recorded in each `main_launch.sh`.

- **Aggregation:** run

```
python3 scripts/prepare_paper01_data.py
--run-root results/paper01_ai_recipe_mutation/runs
--out-dir results/paper01_ai_recipe_mutation
--manifest experiments/manifests/paper01_preprint.yaml
```

to regenerate `validity_report.md`, `tables/run_level.csv`, and the per-RQ summary tables cited in §6. For the statistical numbers (Mann–Whitney U, A_{12}, bootstrap CIs, time-to-N-edges, and the leave-one-out throughput-parity check), run the deterministic script `scripts/paper01/review_stats.py`.

Outside the scope of this preprint, deferred to a follow-up revision: a `controller-only` ablation (plateau detector + corpus snapshot only, no LLM, no recipe-guided mutator, no Ghidra) to close the attribution argument in §6.5; $N \geq 20$ main / $N \geq 10$ ablation repeats with formal Mann–Whitney U significance testing per Klees et al. [10]; the libxml2 / sqlite3 / openssl_x509 multi-target matrix (§8); and a head-to-head measurement against G^{2}FUZZ on its binary-format benchmark. The preprint intentionally limits empirical claims to what the 25 cJSON runs (plus E3 microbench) support.

References

[1] Anonymous. MALF: A multi-agent LLM framework for intelligent fuzzing of industrial control protocols. *arXiv preprint arXiv:2510.02694*, 2025.

[2] Anonymous. Semantic-aware fuzzing: An empirical framework for llm-guided, reasoning-driven input mutation. *arXiv preprint arXiv:2509.19533*, 2025.

[3] Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*, pages 1–10, 2011.

[4] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. NAUTILUS: Fishing for deep bugs with grammars. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '19)*, 2019.

[5] Cornelius Aschermann, Sergej Schumilo, Tim Blazytko, Robert Gawlik, and Thorsten Holz. REDQUEEN: Fuzzing with input-to-state correspondence. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '19)*, 2019.

[6] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*, pages 423–435, 2023.

[7] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. Large language models are edge-case generators: Crafting unusual programs

for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*, 2024.

[8] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. USENIX Workshop on Offensive Technologies (WOOT), 2020. Software v4.21c used in this paper.

[9] Jie Hu, Qian Zhang, and Heng Yin. ChatFuzz: Augmenting greybox fuzzing with large language models. *arXiv preprint arXiv:2308.11525*, 2023.

[10] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*, pages 2123–2138, 2018.

[11] laf intel. Circumventing fuzzing roadblocks with compiler transformations. https://lafintel.wordpress.com, 2016. Blog post, August 15, 2016. The `laf-intel` LLVM pass; integrated into AFL++ as `AFL_LLVM_LAF_*` build flags.

[12] Shiyin Lin. Hybrid fuzzing with LLM-guided input mutation and semantic feedback. *arXiv preprint arXiv:2511.03995*, 2025.

[13] Dongge Liu, Jonathan Metzman, Oliver Chang, and Google OSS-Fuzz team. OSS-Fuzz-Gen: An open framework for LLM-driven fuzz target generation. *arXiv preprint arXiv:2404.14924*, 2024.

[14] Liqun Liu et al. FuzzCoder: Byte-level fuzzing test via large language model. *arXiv preprint arXiv:2409.01944*, 2024.

[15] Conghua Ou et al. The Mutators reloaded: Fuzzing compilers with large language model generated mutators. In *Proceedings of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024.

[16] Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Răzvan Caciulescu, and Abhik Roychoudhury. Smart greybox fuzzing. In *IEEE Transactions on Software Engineering (TSE)*, 2021. Originally NDSS 2019 (AFLSmart); journal extension 2021.

[17] Donald J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.

[18] Yuyan Sun et al. Sphinx: A language model guided generator for solver fuzzing. In *Proceedings of the International Conference on Automated Software Engineering (ASE '24)*, 2024. Representative "LLM upfront, no steady-state calls" design for SMT-solver fuzzing.

[19] András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

[20] Hongxiang Wang, Xiangwei Xu, Xiaofei Xie, et al. LLAMAFUZZ: Large language model enhanced greybox fuzzing. *arXiv preprint arXiv:2406.07714*, 2024. Latest revision March 2026.

[21] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-aware greybox fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE '19)*, 2019.

[22] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer Berlin Heidelberg, 2012.

[23] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. *arXiv preprint arXiv:2308.04748*, 2024. Originally posted 2023; updated 2024.

[24] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. WhiteFox: White-box compiler fuzzing empowered by large language models. In *Proceedings of the ACM on Programming Languages (OOPSLA '24)*, 2024.

[25] Chenyuan Yang, Zhouruixing Zhao, and Lingming Zhang. KernelGPT: Enhanced kernel fuzzing via large language models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*, 2025.

[26] Yupeng Yang, Shenglong Yao, Jizhou Chen, and Wenke Lee. Hybrid language processor fuzzing via LLM-based constraint solving. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, 2025. Georgia Institute of Technology.

[27] J. Ye et al. Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports. *arXiv preprint arXiv:2507.19275*, 2025.

[28] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. Low-cost and comprehensive non-textual input fuzzing with LLM-synthesized input generators. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*, pages 6999–7018, 2025. Also arXiv:2501.19282; the legacy bib key `liu2025g2fuzz` is preserved for citation stability.